

Úvod do jazyka Python

Vybraná témata

Vladimír Župka 2021

Použité zdroje informací.....	5
Kódování znaků Unicode	6
Převody znaků a pořadových čísel Unicode	6
Vybrané řetězcové metody.....	8
Hledání podřetězce – metoda index()	8
Dělení řetězce do seznamu – metoda split()	9
Spojení řetězce s prvky posloupnosti – metoda join()	9
Metody testování řetězce	10
Odstranění konců řetězce.....	10
Náhrada podřetězce.....	10
Hledání maxima čísel v textovém souboru	11
Regulární výrazy.....	12
Funkce match()	12
Celé číslo bez znaménka.....	13
Celé číslo s nepovinným znaménkem.....	13
Funkce search().....	14
Funkce findall().....	14
Funkce finditer()	14
Zlomkové číslo typu float (bez exponentové notace).....	15
Kontrola podtržítka v celém čísle	16
Podtržítka ve zlomkovém čísle	16
Proměnné v regulárním výrazu.....	16
Rozdělení textu zadanými znaky	17
Hledání maxima čísel v textu.....	17
Více o funkcích.....	18
Vícenásobný parametr (variadic parameter).....	18
Kombinace jednoduchého a vícenásobného parametru.....	18
Vícenásobné klíčové parametry (variadic keyword parameters).....	19
Speciální parametry	20
Funkce jsou objekty	21
Funkce jako proměnná.....	21
Funkce jako parametr.....	21
Funkce jako vracená hodnota	21
Funkce jako člen struktur.....	22
Funkce lambda	22
Lambda funkce jako argument.....	23
Lambda funkce jako návratová hodnota	23
Rekurzivní volání funkce.....	24
Tisk posloupnosti v přímém a obráceném pořadí	24
Výpočet faktoriálu	25
Součet geometrické řady.....	26
Příklad pro srovnání – součet geometrické řady bez rekurze	26
Rekurzivní procházení adresáře.....	27
Iterace, iterátory, generátory	28
Postupné čtení prvků iterátoru	28
Čtení prvků iterátoru v cyklu s testem na symbol konce.....	28
Čtení všech prvků iterátoru v cyklu s testem na výjimku.....	29
Iterátor uvnitř cyklu for-in	29

Spojení sekvencí do n-tic – zip()	30
Produkční výraz a cyklické výpočty	31
Výroba nového seznamu (list comprehension)	31
Výroba nové n-tice s hvězdičkovým výrazem (star expression)	31
Generátorový výraz (generator expression)	32
Příklad – výpočet maxima z čísel v souboru a hlášení chyb v datech	33
Funkce map(), filter(), reduce()	34
Generátorové funkce s výrazem yield	36
Vydávací generátorová funkce	36
Přijímací generátorová funkce	38
Příklad – tisk přijatých hodnot	38
Korutiny – pipeline (potrubí)	39
Příklad – příjem výrazem yield a výdej příkazem send()	39
Příklad – postupné zpracování čísel z textového souboru	41
Objekty a jména	43
Obecné vlastnosti objektů	44
Jména – identifikátory	45
Klíčová slova (keywords)	45
Rezervované třídy jmen	45
Vytváření (definice) jmen	46
Prostory jmen (namespaces)	47
Lokální prostor jmen	47
Nelokální (obalový) prostor jmen	47
Globální prostor jmen – modul	47
Vestavěný prostor jmen (built-in namespace)	47
Příklad	48
Klíčová slova global a nonlocal	49
Jak Python hledá jména	50
Volba jmen	50
Moduly	51
Příklad importu	51
Importovaný modul vypocetni_operace.py	51
Hlavní modul vypocty.py	52
Formy importu	53
Přejmenování importovaného modulu	53
Vyjmenování použitých funkcí	53
Použití hvězdičky pro použití všech funkcí z importu	53
Několik modulů v řádku	53
Získání informací o modulech	53
Jak Python hledá moduly	54
macOS	54
Linux	54
Windows	54
Spuštění modulu z příkazového řádku	55
macOS	55
Linux	55
Windows	55
Třídění (sorting)	56
Třídění sekvencí	56
Třídění slovníku	57
Třídění seznamu na místě	57
České třídění	58
Třídění seznamu podle české abecedy (včetně ch)	58
Třídění slovníku podle české abecedy	58

Národní prostředí (locale).....	59
Nastavení českého prostředí	59
Ukázky výstupu čísel v českém prostředí.....	60
Časové údaje	60
Zjištění dnešního data	60
Zjištění současného časového razítka	60
Zjištění denního času	61
Přičtení časového intervalu.....	61
Stanovení světové časové zóny vzhledem k UTC (Universal Time Coordinated).....	61
Formátovaný čas pro výstup	61

Použité zdroje informací

Česká online učebnice základní: <http://howto.py.cz/>

Anglická online učebnice pokročilá: <https://diveintopython3.net/index.html>

Česká online učebnice pokročilá: https://knihy.nic.cz/files/nic/edice/mark_pilgrim_dip3_ver3.pdf

Algoritmy a datové struktury: https://doc.lagout.org/science/0_Computer%20Science/2_Algorithms/

[Data%20Structures%20and%20Algorithms%20using%20Python%20%5BNecaise%202010-12-21%5D.pdf](https://doc.lagout.org/science/0_Computer%20Science/2_Algorithms/Data%20Structures%20and%20Algorithms%20using%20Python%20%5BNecaise%202010-12-21%5D.pdf)

Kódování znaků Unicode

Převody znaků a pořadových čísel Unicode

K převodům slouží následující funkce.

chr() převod pořadového čísla Unicode na znak

ord() převod znaku na pořadové číslo Unicode

Tabulku znaků a pořadových čísel Unicode můžeme vytisknout následujícím programem. Tiskneme jen část latinky s čísly v šestnáctkovém vyjádření.

```
print("\nZákladní latinka")
for i in range(32, 400):
    print(f" {repr(chr(i)):>2} {hex(ord(chr(i)):>4}", end="")
    if i == 128:
        print("\nZápadoevropská latinka - dodatek")
    if i == 256:
        print("\nLatinka rozšířená - A")
    if i == 384:
        print("\nLatinka rozšířená - B")
    if i % 10 == 0:
        print()
```

Základní latinka

```
' ' 0x20 '!' 0x21 '"' 0x22 '#' 0x23 '$' 0x24 '%' 0x25 '&' 0x26 "'" 0x27 '(' 0x28
')' 0x29 '*' 0x2a '+' 0x2b ',' 0x2c '-' 0x2d '.' 0x2e '/' 0x2f '0' 0x30 '1' 0x31 '2' 0x32
'3' 0x33 '4' 0x34 '5' 0x35 '6' 0x36 '7' 0x37 '8' 0x38 '9' 0x39 ':' 0x3a ';' 0x3b '<' 0x3c
'=' 0x3d '>' 0x3e '?' 0x3f '@' 0x40 'A' 0x41 'B' 0x42 'C' 0x43 'D' 0x44 'E' 0x45 'F' 0x46
'G' 0x47 'H' 0x48 'I' 0x49 'J' 0x4a 'K' 0x4b 'L' 0x4c 'M' 0x4d 'N' 0x4e 'O' 0x4f 'P' 0x50
'Q' 0x51 'R' 0x52 'S' 0x53 'T' 0x54 'U' 0x55 'V' 0x56 'W' 0x57 'X' 0x58 'Y' 0x59 'Z' 0x5a
'[' 0x5b '\\' 0x5c ']' 0x5d '^' 0x5e '_' 0x5f '`' 0x60 'a' 0x61 'b' 0x62 'c' 0x63 'd' 0x64
'e' 0x65 'f' 0x66 'g' 0x67 'h' 0x68 'i' 0x69 'j' 0x6a 'k' 0x6b 'l' 0x6c 'm' 0x6d 'n' 0x6e
'o' 0x6f 'p' 0x70 'q' 0x71 'r' 0x72 's' 0x73 't' 0x74 'u' 0x75 'v' 0x76 'w' 0x77 'x' 0x78
'y' 0x79 'z' 0x7a '{' 0x7b '|' 0x7c '}' 0x7d '~' 0x7e '\x7f' 0x7f '\x80' 0x80
```

Západoevropská latinka - dodatek

```
'\x81' 0x81 '\x82' 0x82
'\x83' 0x83 '\x84' 0x84 '\x85' 0x85 '\x86' 0x86 '\x87' 0x87 '\x88' 0x88 '\x89' 0x89 '\x8a' 0x8a
'\x8b' 0x8b '\x8c' 0x8c
'\x8d' 0x8d '\x8e' 0x8e '\x8f' 0x8f '\x90' 0x90 '\x91' 0x91 '\x92' 0x92 '\x93' 0x93 '\x94' 0x94
'\x95' 0x95 '\x96' 0x96
'\x97' 0x97 '\x98' 0x98 '\x99' 0x99 '\x9a' 0x9a '\x9b' 0x9b '\x9c' 0x9c '\x9d' 0x9d '\x9e' 0x9e
'\x9f' 0x9f '\xa0' 0xa0
'i' 0xa1 'ç' 0xa2 'é' 0xa3 'ı' 0xa4 'ſ' 0xa5 'ı' 0xa6 'ş' 0xa7 'ı' 0xa8 '©' 0xa9 'ª' 0xaa
'«' 0xab '¬' 0xac '\xad' 0xad '@' 0xae '¬' 0xaf '°' 0xb0 '±' 0xb1 '²' 0xb2 '³' 0xb3 '´' 0xb4
'µ' 0xb5 '¶' 0xb6 '·' 0xb7 '¸' 0xb8 '¹' 0xb9 'º' 0xba '»' 0xbb '¼' 0xbc '½' 0xbd '¾' 0xbe
'¿' 0xbf 'À' 0xc0 'Á' 0xc1 'Â' 0xc2 'Ã' 0xc3 'Ä' 0xc4 'Å' 0xc5 'Æ' 0xc6 'Ç' 0xc7 'È' 0xc8
'É' 0xc9 'Ê' 0xca 'Ë' 0xcb 'Ì' 0xcc 'Í' 0xcd 'Î' 0xce 'Ï' 0xcf 'Ð' 0xd0 'Ñ' 0xd1 'Ò' 0xd2
'Ó' 0xd3 'Ô' 0xd4 'Õ' 0xd5 'Ö' 0xd6 '×' 0xd7 'Ø' 0xd8 'Ù' 0xd9 'Ú' 0xda 'Û' 0xdb 'Ü' 0xdc
'Ý' 0xdd 'Þ' 0xde 'ß' 0xdf 'à' 0xe0 'á' 0xe1 'â' 0xe2 'ä' 0xe3 'å' 0xe4 'æ' 0xe5 'æ' 0xe6
'ç' 0xe7 'è' 0xe8 'é' 0xe9 'ê' 0xea 'ë' 0xeb 'ì' 0xec 'í' 0xed 'î' 0xee 'ï' 0xef 'ð' 0xf0
'ñ' 0xf1 'ò' 0xf2 'ó' 0xf3 'ô' 0xf4 'õ' 0xf5 'ö' 0xf6 '÷' 0xf7 'ø' 0xf8 'ù' 0xf9 'ú' 0xfa
'û' 0xfb 'ü' 0xfc 'ý' 0xfd 'þ' 0xfe 'ÿ' 0xff 'Ä' 0x100
```

Latinka rozšířená - A

```
'ā' 0x101 'Ā' 0x102 'ā' 0x103 'Ā' 0x104
'ą' 0x105 'Ć' 0x106 'ć' 0x107 'Ĉ' 0x108 'ĉ' 0x109 'Č' 0x10a 'č' 0x10b 'Č' 0x10c 'č' 0x10d 'Ď' 0x10e
'd' 0x10f 'Ð' 0x110 'đ' 0x111 'Ē' 0x112 'ē' 0x113 'Ĕ' 0x114 'ė' 0x115 'Ė' 0x116 'ė' 0x117 'Ě' 0x118
'ę' 0x119 'Ě' 0x11a 'ě' 0x11b 'Ĝ' 0x11c 'ĝ' 0x11d 'Ğ' 0x11e 'ğ' 0x11f 'Ġ' 0x120 'ġ' 0x121 'Ģ' 0x122
...
```

Latinka rozšířená - B

```
'B' 0x181 'Б' 0x182 'Ḃ' 0x183 'Ḃ' 0x184 'b' 0x185 'Ḅ' 0x186
'Ć' 0x187 'ć' 0x188 'Ḍ' 0x189 'Ḍ' 0x18a 'Ḍ' 0x18b 'Ḍ' 0x18c 'Ḍ' 0x18d 'Ḍ' 0x18e 'Ḍ' 0x18f
...
```

Vytiskneme informace o některých znacích Unicode v rozsahu určeném šestnáctkovými čísly. Použijeme formátovací typy x, d.

```
import unicodedata

print("Řecká písmena")
for i in range(0x03b1, 0x03ca):
    print(f"{chr(i)} U+{ord(chr(i)):x}, {ord(chr(i)):d}"
          f", {unicodedata.category(chr(i))}"
          f", {unicodedata.name(chr(i))}")
```

Řecká písmena

α	U+3b1, 945, Ll, GREEK SMALL LETTER ALPHA
β	U+3b2, 946, Ll, GREEK SMALL LETTER BETA
γ	U+3b3, 947, Ll, GREEK SMALL LETTER GAMMA
δ	U+3b4, 948, Ll, GREEK SMALL LETTER DELTA
ε	U+3b5, 949, Ll, GREEK SMALL LETTER EPSILON
ζ	U+3b6, 950, Ll, GREEK SMALL LETTER ZETA
η	U+3b7, 951, Ll, GREEK SMALL LETTER ETA
θ	U+3b8, 952, Ll, GREEK SMALL LETTER THETA
ι	U+3b9, 953, Ll, GREEK SMALL LETTER IOTA
κ	U+3ba, 954, Ll, GREEK SMALL LETTER KAPPA
λ	U+3bb, 955, Ll, GREEK SMALL LETTER LAMDA
μ	U+3bc, 956, Ll, GREEK SMALL LETTER MU
ν	U+3bd, 957, Ll, GREEK SMALL LETTER NU
ξ	U+3be, 958, Ll, GREEK SMALL LETTER XI
ο	U+3bf, 959, Ll, GREEK SMALL LETTER OMICRON
π	U+3c0, 960, Ll, GREEK SMALL LETTER PI
ρ	U+3c1, 961, Ll, GREEK SMALL LETTER RHO
ς	U+3c2, 962, Ll, GREEK SMALL LETTER FINAL SIGMA
σ	U+3c3, 963, Ll, GREEK SMALL LETTER SIGMA
τ	U+3c4, 964, Ll, GREEK SMALL LETTER TAU
υ	U+3c5, 965, Ll, GREEK SMALL LETTER UPSILON
φ	U+3c6, 966, Ll, GREEK SMALL LETTER PHI
χ	U+3c7, 967, Ll, GREEK SMALL LETTER CHI
ψ	U+3c8, 968, Ll, GREEK SMALL LETTER PSI
ω	U+3c9, 969, Ll, GREEK SMALL LETTER OMEGA

```
print("Zvířata")
for i in range(0x1f430, 0x1f438):
    print(f"{chr(i)} - U+{ord(chr(i)):x}, {ord(chr(i)):d}"
          f", {unicodedata.category(chr(i))}"
          f", {unicodedata.name(chr(i))}")
```

Zvířata

	U+1f430, 128048, So, RABBIT FACE
	U+1f431, 128049, So, CAT FACE
	U+1f432, 128050, So, DRAGON FACE
	U+1f433, 128051, So, SPOUTING WHALE
	U+1f434, 128052, So, HORSE FACE
	U+1f435, 128053, So, MONKEY FACE
	U+1f436, 128054, So, DOG FACE
	U+1f437, 128055, So, PIG FACE

Vybrané řetězcové metody

Funkce určené pro řetězce se používají s *tečkou* mezi objektem a jménem funkce. Takové funkci se říká *metoda*, protože pochází z definice třídy (class), kde jsou funkce konstruovány speciálním způsobem a nazývají se metody. Řetězec, tedy objekt typu `str`, je produktem třídy (class) `str`.

Pojem třídy je vysvětlen v jedné z dalších částí kurzu, zatím se bez něj obejdeme.

Hledání podřetězce – metoda `index()`

```
index(value[, start=0[, end=len(value)])
```

Vrací *index* začátku nalezené hodnoty *value* (zadaného řetězce). Nenajde-li *value*, generuje výjimku `ValueError`. Metoda `index()` dovoluje vyhledávat hodnotu nejen v řetězci, ale také v seznamu (a jiných indexovatelných objektech).

```
ret = """My tři králové jdeme k vám, štěstí, zdraví vinšujem vám.  
Štěstí, zdraví, dlouhá léta, my jsme k vám přišli z daleka."""  
  
print(ret.index('tři')) # 1
```

Hledání všech výskytů vzorku bez ohledu na velikost písmen. Metoda `upper()` převádí všechna písmena na velká, metoda `lower()` je převádí na malá.

```
si = [] # seznam nalezených indexů  
sp = [] # seznam nalezených podřetězců  
vzorek = "ŠTĚSTÍ"  
index, start = 0, 0  
while True:  
    try:  
        index = ret.upper().index(vzorek, start)  
        si.append(index)  
        sp.append(ret[index: index + len(vzorek)])  
        start = index + len(vzorek)  
    except ValueError:  
        break  
print(si) # [28, 58]  
print(sp) # ['štěstí', 'štěstí']
```

Výraz `ret.upper().index(vzorek, start)` řetězí dvě metody. Metoda `upper()` aplikovaná na objekt `ret` vrací objekt typu `str` a na něj je aplikovaná metoda `index()`, která vrací objekt typu `int`.

Poznámka: Stejný tvar parametrů má metoda `find()`, která však vrací číslo -1, jestliže se hledaný řetězec nenajde.

Dělení řetězce do seznamu – metoda `split()`

```
split(sep=None, maxsplit=-1)
```

Vrací *seznam* oddělených podřetězců. Oddělovač `sep` může být řetězec, kterým se rozděljuje zadaný řetězec do podřetězců. Podřetězce jsou pak prvky výsledného seznamu. Není-li parametr `sep` zadán (je `None`), dělí se řetězec podle mezer, přesněji podle znaků "whitespace". Parametr `maxsplit` určuje maximální počet dělení, `-1` znamená neomezený počet.

```
st = """jedna    dvě tři
      čtyři pět"""
s = st.split()
print(s)  # ['jedna', 'dvě', 'tři', 'čtyři', 'pět']

st = "jedna ,      dvě,tři      , čtyři,pět"
s = st.split(",")
print(s)  # ['jedna ', '      dvě', 'tři      ', ' čtyři', 'pět']

st = "aaaTRbbbTRccc"
s = st.split(sep='TR')
print(s)  # ['aaa', 'bbb', 'ccc']
```

Spojení řetězce s prvky posloupnosti – metoda `join()`

```
join(iterable)
```

kde `iterable` je posloupnost schopná opakovací operace, např. seznam, n-tice nebo slovník. Vrací *nový řetězec* vytvořený ze spojení zadaného řetězce a prvků posloupnosti. Nejčastěji se používá ke spojení prvků oddělovacími čárkami a mezerou.

```
st = ", ".join(['a', 'b', 'c', 'd'])  # spojí čárku a mezeru s prvky seznamu
print(st)  # a, b, c, d

st = ''.join(('AAA', 'BBB', 'CCC', 'DDD'))  # spojí prázdný znak s prvky n-tice
print(st)  # AAABBBCCCDDD

d = {"A": 1, "B": 2, "C": 2, "D": 3}  # spojí pomlčku s klíči slovníku
st = '-'.join(d)
print(st)  # A-B-C-D

# funkce iter() promění řetězec na "iterable"
print(", " . join(iter('abcd')))  # a, b, c, d
```

Metody testování řetězce

isdigit()

vrátí `True`, jestliže řetězec obsahuje samé číslice.

isspace()

vrátí `True`, jestliže řetězec obsahuje samé bílé znaky (whitespace):

```
' '      0x20   mezera
'\n'    0xa    nový řádek
'\t'    0x9    tabulátor
'\r'    0xd    návrat vozíku (carriage return)
'\v'    0xb    vertikální tabulátor
'\f'    0xc    posuv formuláře (form feed)
```

Odstranění konců řetězce

strip([chars])

Metoda vrátí nový řetězec tak, že z daného řetězce odstraní z obou konců všechny znaky zadané v nepovinném parametru *chars*. Není-li parametr zadán, odstraní bílé znaky (whitespace).

```
print(repr("  nemezery  ".strip())) # 'nemezery'
print(repr("/*.holý text*".strip("*/")) # 'holý text'
print(repr(" /*.holý text*".strip("*/")) # ' /*.holý text'
```

Stejně fungují podobné metody:

lstrip([chars])	odstraňuje znaky ze začátku řetězce (zleva)
rstrip([chars])	odstraňuje znaky z konce řetězce (zprava)

Náhrada podřetězce

replace(old, new[, count])

Metoda vrátí nový řetězec, kde všechny výskyty podřetězce *old* jsou nahrazeny řetězcem *new*. Je-li zadán počet *count*, nahradí se jen první výskyty v počtu *count*.

```
print('ABCD---ABCD---'.replace('---', '**', 1)) # ABCD**ABCD---
```

Hledání maxima čísel v textovém souboru

Textový soubor `cisla.txt` obsahuje v řádcích čísla ve tvaru řetězců oddělených aspoň jednou čárkou a případnými mezerami. Program čte soubor po řádcích metodou `readline()` v cyklu `while` a každý řádek rozdělí do seznamu odstraněním čárky metodou `split(",")`. Metoda `readline()` v přečtených řádcích ponechává koncový znak nového řádku `"\n"`, kterého se musíme zbavit, aby nezpůsobil chybu. Chyby ohlásíme v bloku `try-except`.

```
with open("cisla.txt", "w+") as cisla:
    cisla.write("""23.5, +32,
4_3.5_55,20, *19,
6,4_2""")
    cisla.seek(0) # vrátí se na začátek souboru
    maximum = float('-inf') # nejmenší možné číslo
    index = 0
    line = cisla.readline()
    if not line:
        print("Prázdný vstup")
        exit(2)
    while line:
        print(line, end="")
        ssplit = line.split(",") # rozdělí řádek podle čárek na seznam
        for n in range(len(ssplit)): # převod prvků seznamu na čísla
            try:
                if ssplit[n] != "\n": # vynechávám znak nový řádek
                    fl = float(ssplit[n]) # převod řetězce na číslo
                    if fl > maximum: # výpočet maxima
                        maximum = fl
            except Exception as err:
                print(f"Chyba v {index + 1}. "
                    f"řádku {n + 1}. prvku {repr(ssplit[n])}: {err}")
        line = cisla.readline()
        index += 1
    print("\nmaximum =", maximum)
```

```
23.5, +32,
4_3.5_55,20, *19,
Chyba v 2. řádku 3. prvku ' *19': could not convert string to float: ' *19'
6,4_2
maximum = 43.555
```

Regulární výrazy

Regulární výraz (regular expression, regex, RE) je předpis nebo vzor (pattern) určující požadavky na kontrolovaný vstupní text. Regulární výraz se používá k analýze textu několika způsoby.

- zjišťuje, zda vstupní text vyhovuje zadanému regulárnímu výrazu
- zjišťuje pozice ve vstupním textu, kde je shoda se zadaným regulárním výrazem
- určuje řetězce vyhovující ve vstupním textu regulárnímu výrazu a ukládá je do zvolených proměnných

Práce s regulárními výrazy je v Pythonu umožněna *modulem* nazvaným **re**. Téma je velmi rozsáhlé, proto se spokojíme jen s nejdůležitějšími prostředky.

Regulární výraz se zapisuje jako řetězcový literál v uvozovkách (dvojitých nebo jednoduchých). Může se skládat ze *znaků nebo řetězců*, které musí vstupní text obsahovat, a ze *speciálních znaků a symbolů*. *Zpětné lomítko* před určitými znaky mění jejich význam a dělá z nich speciální symbol, například zpětné lomítko udělá z písmena `d` speciální symbol `\d`.

Zpětné lomítko ve speciálních symbolech regulárního výrazu znamená něco jiného než zpětné lomítko v řetězcovém literálu. Například hledání zpětného lomítka `'\\'` v textu by znamenalo zdvojený zápis `'\\\\'` v regulárním výrazu. Pro takové případy se doporučuje *vždy* zapisovat předponu `r` (raw string, surový řetězec), který tuto nutnost odstraňuje: zpětné lomítko bychom hledali výrazem `r'\\'`.

Funkce `match()`

```
match(pattern, string, flags=0)
```

- *pattern* je regulární výraz zapsaný ve formě znakového řetězce
- *string* je kontrolovaný vstupní text
- *flags* je nepovinný a slouží k modifikaci procesu zpracování regulárního výrazu

Metoda vrátí tzv. "match object" neboli objekt shody, jestliže **začátek** textu vyhovuje regulárnímu výrazu. Jinak vrátí `None`.

Celé číslo bez znaménka

Jednoduchý příklad kontroluje text, zda na začátku obsahuje celé číslo.

```
import re

m = re.match(r'\d+', '45') # '\d+' je regulární výraz, '45' je kontrolovaný text
print(m) # <re.Match object; span=(0, 2), match='45'>

m = re.match(r'\d+', '45 ab') # na začátku je číslo, zbylé znaky nevadí
print(m) # <re.Match object; span=(0, 2), match='45'>

m = re.match(r'\d+', 'ab45') # na začátku není číslo
print(m) # None
```

V tomto příkladu regulární výraz obsahuje jen speciální symboly:

`\d` představuje předpis pro přítomnost číslice 0 až 9.

`+` (znaménko plus) znamená, že v textu se číslice musí vyskytnout alespoň jednou.

Celé číslo s nepovinným znaménkem

Zůstaneme u celých čísel. Připustíme celé číslo s případným znaménkem `+` nebo `-`.

```
print(re.match(r'[+-]?\d+', '-12')) # text je číslo (v pozicích 0 až 2)
<re.Match object; span=(0, 3), match='-12'>
```

V tomto příkladu jsou součástí regulárního výrazu i dva obyčejné textové znaky `+` a `-`. Další znaky jsou speciální, jsou vyznačeny červeně a mají následující význam:

`[ ]` každý ze znaků zapsaných uvnitř hranatých závorek (`+` i `-`) je přípustný na této pozici

`?` znamená, že znak předepsaný před ním (`+` nebo `-`) je nepovinný

Poznámka: V hranatých závorkách mohou být zapsány nejen jednotlivé znaky, ale také rozsahy znaků, např. zápis `[0-9]` znamená `[0123456789]`, a tedy totéž co symbol `\d`.

Funkce `search()`

```
search(pattern, string, flags=0)
```

Slouží k vyhledání prvního podřetězce předepsaného regulárním výrazem. Má stejné parametry jako funkce `match()`, ale chová se jinak. *Hledá* totiž v kontrolovaném vstupním textu od začátku, a teprve když **najde první** předepsaný výsledek, vrátí "match object" neboli objekt shody. Jinak vrátí `None`.

```
print(re.search(r'[+-]?\d+', 'č-12 -12 ')) # číslo -12 je v pozicích 1 až 3  
<re.Match object; span=(1, 4), match='-12'>
```

Funkce `findall()`

```
findall(pattern, string, flags=0)
```

Najde **všechny** předepsané podřetězce. Má opět stejné parametry jako funkce `search()`, ale vyhledává od začátku všechny výskyty předepsaného podřetězce. Najde-li nějaké, vrátí je ve formě *seznamu*. Nenažde-li žádný, vrátí *prázdný seznam* `[]`.

```
print(re.findall(r'[+-]?\d+', ' č-12 -13 abc 14'))  
['-12', '-13', '14']
```

Funkce `finditer()`

```
finditer(pattern, string, flags=0)
```

Vrací *iterátor* vytvořený z nalezených výskytů. Z prvků iterátoru lze získat různé hodnoty. Funkce **group()** dodá nalezený řetězec (hodnotu skupiny), funkce **start()** dodá pozici ve vstupním řetězci. Další metody jsou uvedeny v dokumentaci.

```
iterator = re.finditer(r'[+-]?\d+', ' č-12 -13 abc 14')  
for c in iterator:  
    print(f"číslo:{c.group()}", end=" ")  
    print(f"pozice:{c.start()}", end=" ")  
    print()  
číslo:-12 pozice:2  
číslo:-13 pozice:6  
číslo:14 pozice:14
```

Zlomkové číslo typu **float** (bez exponentové notace)

Budeme potřebovat další speciální symboly:

- * (hvězdička) znamená, že předchozí symbol nebo regulární výraz (zde `\d`) se může libovolně opakovat nebo se nemusí vůbec vyskytnout.
- () (kulaté závorky) pro seskupení regulárních výrazů do jednoho (do skupiny). Se skupinou se pak nakládá jako se samostatným výrazem.
- | znamená alternativu "nebo" (tj. buď + nebo –).

Poloha desetinné tečky ve zlomkovém čísle může být trojí:

- mezi číslicemi `11.11`
- za číslicemi `11.`
- před číslicemi `.11`

Toto pravidlo zkusíme vyjádřit regulárním výrazem. K tomu sestojíme dva podvýrazy, které pak spojíme relací "nebo". První výraz je `\d*[.] \d+` a znamená: nepovinný libovolný počet číslic před tečkou a aspoň jedna číslice za tečkou pro dva výše uvedené případy `11.11` a `.11`. Třetí případ `11.` je vyjádřen výrazem `\d+[.]`: aspoň jedna číslice před tečkou. Oba výrazy spojíme symbolem alternativy (svislou čárkou) a uzavřeme do závorek (aby se k oběma vztahovalo nepovinné znaménko): `(\d*[.] \d+ | \d+[.] )`.

```
print(re.search(r'[+-]?(\d*[.] \d+ | \d+[.] )', '+.11'))  
<re.Match object; span=(0, 4), match='+.11'>
```

```
print(re.search(r'[+-]?(\d*[.] \d+ | \d+[.] )', '-11.'))  
<re.Match object; span=(0, 4), match='-11.'>
```

```
print(re.search(r'[+-]?(\d*[.] \d+ | \d+[.] )', '+11.11'))  
<re.Match object; span=(0, 6), match='+11.11'>
```

Kontrola podtržítka v celém čísle

V celé části i ve zlomkové části čísla smějí být zapsána *podtržítka*, ale mezi dvěma číslicemi jen *jedno*. Například pro jednoduchost celé číslo 1_000. Ke kontrole bychom mohli zkusit výraz `\d+_?\d+`, ale ten nepřipustí jednociferné číslo. Znak `+` zde znamená alespoň jeden výskyt, a ve výrazu je dvakrát, takže číslo by muselo obsahovat aspoň dvě číslice. K nápravě použijeme *kulaté závorky*, které umožňují seskupovat regulární výrazy a se skupinami zacházet jako s jednotkou.

Lepší je výraz `\d(?:\d)*`, kde uvnitř závorek je předepsána posloupnost nepovinného podtržítka a povinné číslice. Hvězdička za závorkou znamená, že obsah závorek se může libovolně opakovat (také vůbec). Výraz `\d` před závorkou značí jednu číslici, která musí být přítomna, čili máme předepsanou aspoň jednu číslici na začátku, která může být následovaná libovolným počtem číslic s *případným* předřazeným podtržítkem.

Funkce `fullmatch()` požaduje úplnou shodu vstupního textu s regulárním výrazem.

```
print(re.fullmatch(r'[+-]?\d(?:\d)*', '-1_000')) # správně umístěné podtržítko
<re.Match object; span=(0, 6), match='-1_000'>

print(re.fullmatch(r'[+-]?\d(?:\d)*', '1000')) # podtržítko není povinné
<re.Match object; span=(0, 4), match='1000'>

print(re.fullmatch(r'[+-]?\d(?:\d)*', '-_1000')) # podtržítko mimo dvě číslice
None
```

Podtržítka ve zlomkovém čísle

```
print(re.search(r'[+-]?(\d*[.]\d+|\d+[.])', '+11.11'))
```

V tomto výrazu pro zlomkové číslo bez podtržítok jsme použili výrazy `\d*` a `\d+`. Oba teď nahradíme výrazem `\d(?:\d)*`, který zahrnuje nepovinné podtržítko.

```
print(re.search(r'[+-]?(\d(?:\d)*[.]\d(?:\d)*|(\d(?:\d)*[.]))', '+2_300.50'))
<re.Match object; span=(0, 9), match='+2_300.50'>
```

Proměnné v regulárním výrazu

Výraz `\d(?:\d)*` si můžeme uložit do proměnné a tu pak několikrát použít ve složeném regulárním výrazu.

```
c = r"\d(?:\d)*"
print(re.search(rf'[+-]?({c}[.]{c})|({c}[.])?', '+2_300.50'))
<re.Match object; span=(0, 9), match='+2_300.50'>
```

Na konec výrazu jsme ještě přidali otazník, aby tečka na konci čísla nebyla povinná, takže projde i celé číslo.

Rozdělení textu zadanými znaky

Funkce `split()` vyjme z textu podřetězce, které jsou rozděleny zadanými znaky, a umístí je do seznamu. Tato funkce umožňuje zadat *několik* jednotlivých oddělovacích znaků, *na rozdíl* od metody `split()` z třídy `str`, která zadává jeden řetězec jako oddělovač.

V příkladu je zadán řetězec, který je členěn do řádků. Stejně jako v příkladu z předchozí kapitoly chceme vypočítat maximum z čísel, která získáme rozdělením textu čárkami nebo mezerami (whitespace):

`\s` znamená "bílý znak" (whitespace), tedy `[\t\n\r\f\v]`

```
lines = """23.5, +32,,,
4_3.5_55,20, *19,
6,4_2"""

s = re.split(r'[\s]', lines)
print(s)
['23.5', '', '+32', '', '', '', '', '4_3.5_55', '20', '', '*19', '', '6', '4_2']
```

Ve výsledném seznamu se objevují prázdné znaky na místech, kde oddělovací znak nenašel žádný podřetězec, tedy tam, kde spolu sousedí dva oddělovací znaky. Když za hranatou závorku přidáme symbol `+` pro opakování (jednou nebo vícekrát), stane se ze sousedících oddělovacích znaků jeden oddělovač, který už nenajde žádný prázdný podřetězec.

```
s = re.split(r'[\s]+', lines)
print(s)
['23.5', '+32', '4_3.5_55', '20', '*19', '6', '4_2']
```

Hledání maxima čísel v textu

Z prvků tohoto seznamu pak, podobně jako v příkladu z předchozí kapitoly, vypočítáme maximální číslo, přičemž chybné podřetězce vyloučíme s oznámením chyby.

```
maximum = float('-inf') # nejmenší možné číslo
for n in range(len(s)):
    try:
        fl = float(s[n]) # převod řetězce na číslo
        if fl > maximum: # výpočet maxima
            maximum = fl
    except Exception as err:
        print(f"Chyba v prvku {repr(s[n])}: {err}")
print("maximum =", maximum)
```

```
Chyba v prvku '*19': could not convert string to float: '*19'
maximum = 43.555
```

Více o funkcích

Vícenásobný parametr (variadic parameter)

Je-li jméno parametru v *definici funkce* označené *hvězdičkou*, představuje libovolný (i nulový) počet povinně *pozičních* parametrů. Takových parametrů může být v definici funkce více, ale až za případnými obyčejnými (bezhvězdičkovými) parametry.

```
def func(*par): # libovolný počet hodnot
    print(par) # tiskne n-tici
```

Ve volání funkce mohou být argumenty zapsány jako hodnoty oddělené čárkou (v podobě n-tice) a *nesmějí mít jména*. Nebo mohou být zapsány jako *hvězdičkový výraz* (starred expression), tj. jméno posloupnosti s předřazenou hvězdičkou.

Argumenty v podobě n-tice

```
func(1, 2, "a", "b") # (1, 2, 'a', 'b') -> n-tice
func() # () - prázdná n-tice
```

Argument jako hvězdičkový výraz

```
args = [1, 2, "a", "b"] # seznam
func(*args) # (1, 2, 'a', 'b') -> rozbalená (unpacked) n-tice
func(*range(10)) # (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Kombinace hvězdičkového výrazu a n-tice

```
arg1 = [1, 2]
arg2 = ["a", "b"]
func(*arg1, *arg2) # (1, 2, 'a', 'b') - kombinace -> celá n-tice
func(*arg1, "a", "b") # (1, 2, 'a', 'b') - stejné
```

Argument jako posloupnost bez hvězdičky

```
func(args) # ([1, 2, 'a', 'b'],) - seznam -> jednočlenná n-tice
```

Kombinace jednoduchého a vícenásobného parametru

```
def maximum(nadpis, *cisla):
    print(nadpis, end=' ')
    max = 0
    for n in cisla:
        if n > max: max = n # blok na stejném řádku se záhlavím příkazu if
    return max
```

```
print(maximum("maximum:", 1, 2, 3.0, 4))
maximum: 4
```

Vícenásobné klíčové parametry (variadic keyword parameters)

Je-li jméno parametru v *definici funkce* označené *dvojhvězdičkou*, představuje libovolný (i nulový) počet povinně *klíčových* parametrů. Takový parametr může být v definici funkce jen jeden, a to jako poslední. V definici ovšem mohou být i jiné parametry před ním.

```
def func(**kwpars): # libovolný počet klíčových parametrů
    print(kwpars)
```

Ve volání funkce mohou být zapsány klíčové argumenty s *libovolnými jmény* a hodnotami oddělené čárkou. Nebo mohou být zapsány jako *dvojhvězdičkové* argumenty (jména slovníků s předřazenou dvojhvězdičkou), anebo jejich kombinace.

Tiskne slovník s klíči podle jmen argumentů

```
func(a1=1, a2=2, a3=3, a4=4) # {'a1': 1, 'a2': 2, 'a3': 3, 'a4': 4}
func() # {} # prázdný slovník
```

Dvojhvězdičkový výraz produkuje stejný slovník

```
slovník = {"a": 1, "b": 2, "c": 3, "d": 4}

func(**slovník) # {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Kombinace klíčových argumentů a dvojhvězdičkových argumentů

```
slov = {"c": 3, "d": 4} # částečný slovník
slov2 = {"e": 5, "f": 6} # jiný slovník

func(a1=1, a2=2, **slov, **slov2)
# {'a1': 1, 'a2': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6}
```

Argument jako slovník bez dvojhvězdičky zkopíruje svou hodnotu a uloží ji jako *hodnotu* nového slovníku s jedním klíčem (jménem argumentu).

```
slovník = {"a": 1, "b": 2, "c": 3, "d": 4}

func(cely_slovník=slovník)
# {'cely_slovník': {'a': 1, 'b': 2, 'c': 3, 'd': 4}}
```

Speciální parametry

Znaky `/` a `*` jsou zvláštní parametry, které oddělují a omezují skupiny ostatních parametrů na *povinně poziční, poziční nebo klíčové, povinně klíčové* v následujícím pořadí.

(*poziční* `/`, *poziční-nebo-klíčové* `*`, *klíčové*)

Omezení skupin parametrů používají standardní a obecně používané funkce, aby znemožnily nebo naopak nařídily uživateli použít poziční nebo klíčový argument.

Například sčítací funkce

```
sum(iterable, /, start=0)
```

předepisuje zadávat první argument jako poziční, bez jména parametru, druhý nepovinný argument může být poziční i klíčový.

```
ntice = (3, 2, 1)
print(sum(ntice, 100)) # 106
print(sum(ntice, start=100)) # 106
```

Třídící funkce

```
sorted(iterable, *, key=None, reverse=False)
```

předepisuje první argument jako poziční, a další dva nepovinné argumenty jako povinně klíčové.

```
seznam = ["ccc", "a", "bb"]
print(sorted(seznam, reverse=True)) # ['ccc', 'b', 'aa']
print(sorted(seznam, key=len)) # ['b', 'aa', 'ccc'] (třídí podle délky)
print(sorted(seznam)) # ['aa', 'b', 'ccc']
```

Poznámka: Název *iterable* znamená, že objekt obsahuje více položek, k nimž lze přistupovat postupně, k jednomu po druhém.

Funkce jsou objekty

Funkce je objekt s typem `function`.

```
def fun(x):
    print(x)
fun(10) # 10

print(type(fun)) # <class 'function'> - typ objektu
print(fun) # <function fun at 0x108b5b040> - hodnota objektu
print(id(fun)) # 4426371136
```

Funkce jako proměnná

Funkci můžeme *přiřadit k proměnné*, která tak odkazuje na její objekt.

```
gun = fun # gun je proměnná odkazující na objekt fun
print(gun == fun) # True
gun(10) # 10 - koná stejnou činnost jako fun
print(type(gun)) # <class 'function'>
print(gun) # <function fun at 0x108b5b040> - odkazuje na stejný objekt fun
```

Funkce jako parametr

Funkce může být *parametrem* jiné funkce. Funkce `fun` je parametrem funkce `hun`:

```
def hun(fun):
    fun(20)
hun(fun) # 20
```

Funkce jako vrácená hodnota

Vnitřní funkce může být *vrácena* z vnější funkce příkazem `return`. Pamatuje si hodnoty lokálních proměnných vnější funkce i když jsou mimo definici vnitřní funkce. Lokální proměnné jsou i vstupní argumenty vnější funkce.

```
def vnejsi(s):
    t = 4000
    def vnitřni(x):
        print(s + t + x) # přičítá vstupní argument vnější funkce
        return vnitřni
    print(vnejsi(0)) # <function vnejsi.<locals>.vnitřni at 0x10e28c820>

vnitr = vnejsi(10) # navracená vnitřní funkce se ukládá do proměnné vnitr
vnitr(3) # 4013
vnitr(9) # 4019
```

Funkce jako člen struktur

Funkce může být členem struktur jako je seznam, n-tice apod.

```
def f1():
    print("f1", end=' ')
def f2():
    print("f2", end=' ')
def f3():
    print("f3", end=' ')

sf = [f1, f2, f3] # seznam funkcí
for i in sf: # spuštění funkcí ze seznamu
    i()
f1 f2 f3
```

Funkce lambda

Funkce lambda je bezejmenná (anonymní) funkce. Je definována "lambda výrazem"

```
lambda [parametry] : výraz
```

Parametry se zapisují bez závorek a za dvojtečkou je výraz. Například lambda výraz

```
lambda a, b: a + b
```

generuje funkci se dvěma argumenty, která vrací jejich součet. Takovou funkci lze *vyvolat* v jiném výrazu, například v závorkách a s připojenými hodnotami argumentů.

```
vysledek = (lambda a, b: a + b) (1, 2)
print(vysledek) # 3
```

Lambda výraz je možné zapsat všude, kde se dá zapsat jméno funkce. Používá se nejčastěji jako *argument jiné funkce*, ale také jako *vracená hodnota*. Lambda funkce zapsaná uvnitř vnější funkce má k dispozici všechny její hodnoty (argumentů i lokálních proměnných) stejně jako jiná vnitřní funkce.

Lambda funkce jako argument

Funkce `gun` vrací hodnotu argumentu. Funkce `fun` přijímá funkci `gun` jako parametr a vrací výsledek volání funkce `gun` (číslo 20).

```
def gun(arg):  
    return arg  
  
def fun(gun):  
    return gun(20)
```

Funkci `fun` vyvoláme s argumentem lambda funkce, která zdvojnásobuje svůj argument.

```
result = fun(lambda x: 2*x) # za gun dosadím lambda funkci
```

Výsledek je dvojnásobek čísla 20 (argumentu `x` lambda funkce)

```
print(result) # 40
```

Stejný výsledek dostaneme, když vyvoláme lambda funkci (v závorkách) s argumentem 20

```
print( (lambda x: 2*x)(20) ) # 40
```

Lambda funkce jako návratová hodnota

Funkce `vnejsi` vrací funkci, která je produktem lambda výrazu.

```
def vnejsi(s):  
    t = 4000  
    return lambda x: s + t + x  
  
funkce = vnejsi(s=20) # vrací funkci
```

Volání funkce `vnejsi` vrátilo funkci. Tu můžeme vyvolat s jedním argumentem a dostaneme číselný výsledek.

```
print( funkce(1) ) # 4021
```

Stejný výsledek dostaneme, když volání `vnejsi(20)` vyvoláme s argumentem 1

```
print( vnejsi(20)(1) ) # 4021
```

Rekurzivní volání funkce

Funkce může uvnitř svého těla (bloku) volat *sama sebe*, což se nazývá *rekurzivní volání funkce*. Takové volání je umožněno tím, že při každém vyvolání se vytvoří nové hodnoty lokálních proměnných a uloží do *zásobníku volání*.

Funkce musí vždy obsahovat nějakou podmínku pro ukončení rekurze, aby sama sebe nevolala nekonečně dlouho. Kdyby podmínka chyběla, volání by se opakovalo bez omezení (tomu Python zabránil tím, že umožní nejvýše 1000 rekurzí).

Jakmile rekurzivní volání skončí, začnou se provádět další výrazy a příkazy s hodnotami vybranými v obráceném pořadí ze zásobníku a ovšem také návraty z vnořených volání funkce.

Tisk posloupnosti v přímém a obráceném pořadí

Vytiskneme posloupnost prvních přirozených čísel dvěma funkcemi. První vytiskne čísla v *sestupném* pořadí, tedy od největšího do nejmenšího. Druhá vytiskne tutéž posloupnost v *obráceném* pořadí.

```
def prvni(n) :  
    print(n)  
    if n > 0 :  
        prvni(n-1)
```

```
prvni(3) # první volání
```

```
def druha(n) :  
    if n > 0 :  
        druha(n-1)  
    print(n)
```

```
druha(3) # první volání
```

Zatímco první funkce tiskne čísla 3 2 1 0, druhá tiskne čísla 0 1 2 3. Obě funkce volají samy sebe, vypadají skoro stejně, ale liší se umístěním příkazu `print(n)`.

n	příkaz
3	prvni (3) první volání
3	print(3)
2	prvni (2)
2	print(2)
1	prvni (1)
1	print(1)
0	prvni (0) poslední volání

n	příkaz
3	druha (3) první volání
2	druha (2)
1	druha (1)
0	druha (0) poslední volání
0	print(0)
1	print(1)
2	print(2)
3	print(3)

Obě funkce volají samy sebe s argumentem vždy o 1 menším než minule, až do nuly.

V první funkci se před voláním hned tiskne číslo `n`, i když se také ukládá do zásobníku volání. Po skončení rekurze, tj. po posledním volání funkce sama sebe, v zásobníku zůstávají čísla `n`, `n-1`, ..., `0`, ale pak už nenásleduje žádný příkaz a výpočet končí.

Ve druhé funkci se začnou tisknout hodnoty brané ze zásobníku (v obráceném pořadí), ale až po skončení rekurzivního volání.

Výpočet faktoriálu

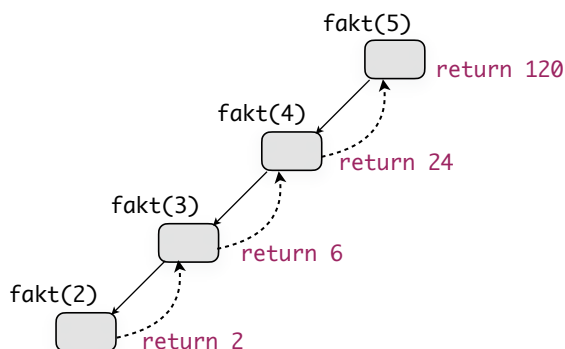
Faktoriál přirozeného čísla n je definován součinem $n \cdot (n - 1) \cdot (n - 2) \dots 2 \cdot 1$ a označuje se symbolem $n!$. Dá se také vyjádřit rekurentním vzorcem $n! = n \cdot (n - 1)!$ pro $n > 0$.

Funkci sestavíme podle rekurentního vzorce pro $n > 2$. Jednoduché případy $1! = 1$, $2! = 2$ můžeme vynechat.

```
def fakt(n):  
    if n > 2: # ukončovací podmínka  
        n = n * fakt(n-1) # ukládání n do zásobníku  
    return n # postupné vracení hodnot
```

```
print("5! =", fakt(n=5)) # 5! = 120
```

n	příkaz	
5	fakt(5)	první volání funkce (zvenčí)
4	fakt(4)	
3	fakt(3)	
2	fakt(2)	poslední vnořené volání (3-1 není větší než 2)
2	return 2	z konce zásobníku
3	return 6	z výpočtu 3 x 2
4	return 24	z výpočtu 4 x 6
5	return 120	z výpočtu 5 x 24



Poznámka: Výpočet faktoriálu se uvádí pro znázornění mechanismu rekurze. Obvykle se však faktoriál počítá obyčejným cyklem, který je rychlejší.

```
def fakt(n):  
    k = 1  
    for i in range(n, 1, -1): # i = n, ..., 2  
        k = k * (i - 1)  
    return(k)
```

```
print("5! =", fakt(n=5)) # 5! = 120
```

Součet geometrické řady

Spočítáme částečný součet geometrické řady, tj. součet k prvních členů geometrické posloupnosti q^n , kde $n \geq 0$, tedy $s = \sum_{n=0}^k q^n$.

```
def sum_geom(q, k, s):  
    if k < 0: # podmínka pro ukončení rekurze  
        return s  
    s += q ** k # během rekurze se přičítají mocniny  
    return sum_geom(q, k-1, s) # až po skončení rekurze se vrátí součet  
  
print("suma = ", sum_geom(q=1/2, k=20, s=0)) # suma = 1.9999990463256836
```

Příklad pro srovnání – součet geometrické řady bez rekurze

Vzorec $s = \sum_{n=0}^k q^n$ lze rozepsat takto: $s = q^0 + q^1 + \dots + q^k$.

```
def sum_geom(q, k, s): # od 0 do k  
    for n in range(0, k+1):  
        s += q ** n  
    return s  
  
print("suma =", sum_geom(q=1/2, k=20, s=0)) # suma = 1.9999990463256836
```

Operace umocňování je výpočetně náročná, proto se nahrazuje postupným násobením.

```
def sum_geom(q, k, s):  
    for n in range(0, k+1): # od 0 do k  
        s = 1 + s * q  
    return s  
  
print("suma =", sum_geom(q=1/2, k=20, s=0)) # suma = 1.9999990463256836
```

Poznámka: Nekonečná řada $s = \sum_{n=0}^{\infty} q^n$ má součet $\frac{1}{1-q}$ pro $|q| < 1$. Pro $q = \frac{1}{2}$ je to číslo 2.

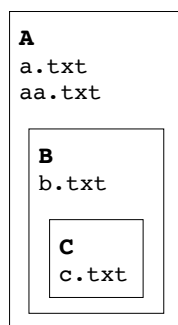
Rekurzivní procházení adresáře

Pro výpis obsahu adresáře je zapotřebí rekurze, protože jde o hierarchickou strukturu s předem neznámým počtem úrovní. Použijeme k tomu modul nazvaný **pathlib**.

```
from pathlib import Path # modul pathlib pro práci s cestami
```

Nejdříve vytvoříme tři vnořené adresáře a v nich soubory.

```
pathd = Path('A/B/C')
try:
    pathd.mkdir(parents=True)
except:
    pass # když už existují
pathf = Path('A/a.txt')
pathf.open(mode='w')
pathf = Path('A/aa.txt')
pathf.open(mode='w')
pathf = Path('A/B/b.txt')
pathf.open(mode='w')
pathf = Path('A/B/C/c.txt')
pathf.open(mode='w')
```



Vypíšeme jména všech souborů a podadresářů v adresáři **A** *rekurzivním* způsobem. K vytvoření seznamu objektů v jednom adresáři použijeme metodu **iterdir()**. Zde je ukončení rekurze dáno konem iterace v metodě **iterdir()**.

```
def obsah_adresare(level, path):
    dir = Path(path) # vytvoří se objekt typu Path
    for elem in dir.iterdir(): # prochází zadaný adresář
        if elem.is_dir(): # je prvek adresáře adresář?
            print(f"{'-'*level}Dir:", elem.name)
            level += 1
            obsah_adresare(level, elem.path) # rekurze
            level -= 1
        else: # jinak prvek je soubor
            print(f"{'-'*level}File:", elem.name)
```

```
obsah_adresare(level=0, path='A'). # procházíme adresář A v běžném adresáři
```

```
File: aa.txt
File: a.txt
Dir: B
-File: b.txt
-Dir: C
--File: c.txt
```

Poznámka: Za argument volání můžeme zvolit libovolnou cestu.

Iterace, iterátory, generátory

Iterace znamená opakování. Vícesložkové objekty, které tvoří *posloupnosti* (sekvence), jsou *opakovatelné*, schopné iterace, anglicky *iterable*. Konkrétně jsou to *seznamy*, *řetězce*, *n-tice*, *slovníky*, *textové soubory s řádky* aj. Proces iterace se uskutečňuje hlavně v opakovacích příkazech `while` a `for-in`. Existují však i jiné prostředky pro iterace, například *generátorové výrazy* (viz dále).

Všechny tyto prostředky využívají tzv. *iterátoru*, což je objekt, který dovoluje získávat následující prvek posloupnosti pomocí funkce `next()`. Iterátor se vytvoří z posloupnosti pomocí funkce `iter()`.

```
s = ["A", "B", "C", "D", "E"]

si = iter(s)          # převod seznamu na iterátor
```

Postupné čtení prvků iterátoru

```
prvek = next(si) # čtu další prvek
print("prv =", prvek) # prv = A
prvek = next(si)
print("prv =", prvek) # prv = B
...
```

Budeme-li pokračovat dále až za poslední prvek "E", nastane chyba `StopIteration`.

Čtení prvků iterátoru v cyklu s testem na symbol konce

Funkce `next()` se dvěma argumenty umožňuje testovat, zda byly přečteny všechny prvky iterátoru. Po přečtení všech prvků vrátí funkce hodnotu druhého argumentu místo dalšího prvku, který neexistuje. Druhý argument si zvolíme, může to být jakýkoliv objekt.

```
textit = iter("abcde") # převod řetězce na iterátor
while True:
    prvek = next(textit, "konec")
    # po vyčerpání všech prvků (znaků) funkce vrátí řetězec "konec"
    if prvek == "konec": # je-li prvek shodný se zadanou hodnotou,
        break          # ukončím cyklus
    else:
        print(prvek, end=" ")

a b c d e
```

Čtení všech prvků iterátoru v cyklu s testem na výjimku

Příkaz `next()` při vyčerpání posloupnosti vygeneruje *výjimku* `StopIteration`. K testování konce seznamu použijeme blok `try-except`.

```
n = ("A", "B", "C", "D", "E") # ntice

ni = iter(n) # převod ntice na iterátor
while True:
    try:
        prvek = next(ni)
        print(prvek, end=" ")
    except StopIteration: # když došlo k výjimce (není další prvek),
        break            # končím cyklus

A, B, C, D, E,

r = range(1, 10, 2)
ri = iter(r) # převod posloupnosti vzniklé z funkce range() na iterátor
while True:
    try:
        prvek = next(ri)
        print("r =", prvek, end=" ")
    except StopIteration:
        break

r = 1, r = 3, r = 5, r = 7, r = 9,
```

Iterátor uvnitř cyklu `for-in`

V cyklu `for-in` není zapotřebí funkce `iter()` ani `next()`. Příkaz `for-in` si *interně* převádí posloupnost na iterátor, používá funkci `next()` a také testuje výjimku `StopIteration`.

```
for prvek in range(1, 10, 2):
    print("r =", prvek, end=" ")

r = 1, r = 3, r = 5, r = 7, r = 9,

for prvek in "abcde":
    print(prvek, end=" ")

a b c d e
```

Spojení sekvencí do n -tic – `zip()`

Funkce `zip()` spojuje stejnohlé prvky z n souběžných posloupností do n -tic a vrací *iterátor*. Funkce prochází posloupnosti jen do vyčerpání té nejkratší.

V následujícím příkladu se spojují stejnohlé prvky *dvou* posloupností do *dvojic*. Číslo 6 se už neuplatní.

```
entice = 1, 2, 3, 4, 5, 6
seznam = ["A", "B", "C", "D", "E"]

for z in zip(entice, seznam):
    print(z, end=", ")

(1, 'A'), (2, 'B'), (3, 'C'), (4, 'D'), (5, 'E'),
```

Produkční výraz a cyklické výpočty

Produkční výraz je zkrácený zápis cyklického výpočtu, který z dané posloupnosti vytváří novou posloupnost.

výstupní-výraz **for** *proměnná* **in** *posloupnost* [**if** *podmínka*]

Produkční výraz se používá v různých souvislostech – vytváří *seznam*, *n-tici*, *slovník* nebo *generátor*.

Výroba nového seznamu (list comprehension)

Použije-li se produkční výraz v *hranatých závorkách*, nazývá se *list comprehension* (generátorový výraz seznamu).

Následující příklad vytvoří z *n-tice* seznam tak, že prohlédne všechny prvky posloupnosti a podrobí je případné podmínce. Vyhoví-li prvek podmínce (že je to celé číslo), vypočítá z něj druhou mocninu a tu zařadí na konec nového seznamu. Výsledkem je tedy *nový seznam*.

```
entice = 1, "a", 3, "c", 5, "e"

s = [x*x for x in entice if type(x) == int]
print(s) [1, 9, 25]
```

Výstupním výrazem je součin `x*x`, proměnnou je `x`, posloupností je `entice` a podmínkou je `type(x) == int`.

Výsledek je jednořádkového příkazu je stejný, jako z následujícího cyklu o pěti řádcích:

```
s = list()
for x in entice:
    if not type(x) == int:
        continue
    s.append(x*x)
print(s) [1, 9, 25]
```

Výroba nové *n-tice* s hvězdičkovým výrazem (star expression)

Použije-li se produkční výraz v závorkách (hranatých, kulatých nebo složených) s předřazenou hvězdičkou a zařazenou čárkou, *vyrobí* z posloupnosti novou *n-tici*. Výraz s úvodní hvězdičkou se nazývá *star expression* (hvězdičkový výraz) a vyskytuje se u rozbalení posloupnosti nebo u parametrů funkce. Koncová čárka je nutná, zajišťuje, že výsledek bude *n-tice*.

```
s = "ABCDE" # řetězec jako posloupnost

n = *[(x+x).lower() for x in s if x < "D"], # "star expression"
print(n) # ('aa', 'bb', 'cc')
```

Generátorový výraz (generator expression)

Použije-li se produkční výraz v *kulatých závorkách*, nazývá se prostě *generátorový výraz*. Ten, místo aby najednou vygeneroval celý objekt obsahující výsledné prvky, vygeneruje objekt zvaný *líný generátor*, což je speciální druh iterátoru. Líný se mu říká, protože nedělá nic, dokud se na něm neprovede první příkaz `next ( )`. Další příkaz `next ( )` pak vyprodukuje další výsledek výstupního výrazu, atd. Vidíme, že generátor je také *iterátor*.

```
entice = 1, "a", 3, "c", 5, "e"
```

```
generator = (x*x for x in entice if type(x) == int)
next(generator) # 1
next(generator) # 9
next(generator) # 25
```

Budeme-li pokračovat dále za poslední prvek posloupnosti, nastane chyba `StopIteration`.

Ze stejného produkčního výrazu můžeme vytvořit dva nebo *více generátorů*, z nichž každý funguje nezávisle, protože si vždy zapamatují naposledy zpracovaný prvek.

```
gen1 = (x*x for x in entice if type(x) == int)
gen2 = (x*x for x in entice if type(x) == int)
next(gen1) # 1
next(gen1) # 9
next(gen2) # 1
```

Použijeme-li generátor s příkazem `next ( )` v cyklu `while`, musíme konec ošetřit sami:

```
generator = (x*x for x in entice if type(x) == int)
while True:
    try:
        print(next(generator), end=", ")
    except StopIteration:
        break
1, 9, 25,
```

Když však generátor použijeme v příkazu `for-in`, nemusíme ani používat příkaz `next ( )`, ani ošetřovat chybu. Ten si vše zařizuje sám.

```
generator = (x*x for x in entice if type(x) == int)
for x in generator:
    print(x, end=", ")
1, 9, 25,
```


Příklad – výpočet maxima z čísel v souboru a hlášení chyb v datech

Generátorový výraz má proti předchozím případům použití produkčního výrazu (třeba list comprehension) velkou výhodu v případech, kdy je posloupnost velmi dlouhá, například při zpracování velkých textových souborů. Vytvořený generátor nezpracuje celý soubor najednou, ale postupně, *řádek po řádku*. Nezaplní tedy tak velký objem operační paměti počítače, jako kdybychom použili třeba *seznam* řádků (list comprehension). Tato výhoda se ovšem ztratí, je-li soubor tvořen jen jedním řádkem.

V příkladu zvolíme vstupní soubor o třech řádcích `cisla.txt`:

```
23.5, 32,  
43.55 ,20, *19,  
6,42.
```

Program kromě výpočtu maxima ještě pro přehled otiskne řádky a ohlásí chyby s číslem řádku a prvku. Pro pouhý výpočet maxima můžeme tyto tisky vynechat.

```
maximum = float('-inf') # nejmenší možné číslo  
# Vytvoříme generátor, který vydává řádky souboru  
# a přitom odstraňuje znaky "whitespace"  
gener = (line.strip() for line in open("cisla.txt"))  
cislo_radku = 0  
for line in gener:  
    print(f"řádek {cislo_radku + 1}: {repr(line)}")  
    # vytvoříme další generátor vydávající znaková čísla  
    # očištěná od čárek a prázdných znaků  
    gen = (zc for zc in line.split(',') if zc != '')  
    prvek = 0  
    for cis in gen:  
        try:  
            fl = float(cis) # převod do čísla typu float  
            if fl > maximum: # výpočet maxima  
                maximum = fl  
        except ValueError as value_error:  
            print(f" Chyba v {cislo_radku + 1}. "  
                  f"řádku {prvek + 1}. prvku {repr(cis)}: {value_error}")  
            continue  
        prvek += 1  
    cislo_radku += 1  
print("maximum =", maximum)
```

```
řádek 1: '23.5, 32,'  
řádek 2: '43.55 ,20, *19,'  
  Chyba v 2. řádku 3. prvku ' *19': could not convert string to float: ' *19'  
řádek 3: '6,42'  
maximum = 43.55
```

Funkce `map()`, `filter()`, `reduce()`

Oblíbené funkce pracující s posloupnostmi jsou vestavěné funkce `map()`, `filter()` a funkce `reduce()`, která je součástí modulu `functools`. Funkce vracejí *iterátor*. Lze je nahradit jinými konstrukcemi nebo cyklem `for-in`.

`map(function, iterable, *iterables)`

Provede funkci `function` se všemi prvky posloupnosti `iterable`. Jsou-li zadány posloupnosti `iterables`, provede funkci i s jejich prvky, funkce musí mít tolik parametrů, kolik je dodatečných posloupností.

```
s = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,  
t = 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
```

Provedení lambda funkce druhé mocniny na jedné posloupnosti:

```
for _ in map(lambda x: x**2, s): # druhé mocniny prvků od 0 do 10  
    print(_, end=', ') # 0, 1, 4, 9, 16, 25, 36, 49, 64, 81,
```

Stejný výsledek s použitím generátorového výrazu seznamu (list comprehension):

```
[ print(x**2, end=', ') for x in s ] # 0, 1, 4, 9, 16, 25, 36, 49, 64, 81,
```

Provedení funkce sčítání odpovídajících prvků ze dvou posloupností:

```
for _ in map(lambda x, y: x + y, s, t):  
    print(_, end=', ') # 10, 12, 14, 16, 18, 20, 22, 24, 26, 28,
```

Stejný výsledek s použitím funkce `zip()`:

```
for x, y in zip(s, t): # spojení do dvojic  
    print(x + y, end=', ') # součet obou prvků dvojice
```

Stejný výsledek s použitím cyklu `for-in`:

```
for i in s:  
    print(s[i] + t[i], end=', ') # 10, 12, 14, 16, 18, 20, 22, 24, 26, 28,
```

`filter(function, iterable)`

Provede funkci *function* se všemi prvky posloupnosti *iterable*. Výsledkem je iterátor, který obsahuje prvky, pro něž je výsledek funkce pravdivý (nenulový, neprázdný apod.), Vynechá tedy z posloupnosti nežádoucí (nulové, prázdné) prvky.

```
s = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
```

```
def f(x):  
    return x % 2 # zbytek modulo 2
```

Provedení funkce zbytku po dělení dvěma (modulo 2), tj. propustí jen lichá čísla, protože sudá dávají nulu:

```
for _ in filter(f, s):  
    print(_, end=' ') # 1 3 5 7 9
```

```
[ print(x, end=' ') for x in s if f(x) ] # totéž s "list comprehension"
```

`functools.reduce(function, iterable[, initializer])`

Provede funkci *function* se dvěma argumenty, např. `fun(x, y)` na všechny prvky posloupnosti *iterable*, např. `s` tak, že výsledek z prvních dvou prvků použije jako první argument a třetí prvek jako druhý argument, atd., čímž zredukuje posloupnost na jednu hodnotu.

$$r_1 = \text{fun}(s[0], s[1]), r_2 = \text{fun}(r_1, s[2]), \dots, r_n = \text{fun}(r_{n-1}, s[n])$$

Je-li zadána hodnota *initializer*, předradí se před prvky posloupnosti a slouží i jako předvolená hodnota pro prázdnou posloupnost.

```
from functools import reduce
```

```
s = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
```

```
reduce(lambda x, y: x + y, s) # 45  
sum([x for x in s]) # 45
```

```
def f(x):  
    return x % 2 # zbytek modulo 2
```

Přičítá k číslu 100 postupně prvky z posloupnosti `s` filtrované funkcí `f`:

```
reduce(lambda x, y: x + y, filter(f, s), 100) # 125  
sum([x for x in filter(f, s)]) + 100 # 125
```

Generátorové funkce s výrazem `yield`

Generátorová funkce (generator function) se vyznačuje tím, že obsahuje výraz se slovem `yield` (poskytnout, vydat). Voláním takové funkce vznikne objekt generátoru. Můžeme rozeznat dvě podoby generátorové funkce: vydávací a přijímací, podle toho na které straně výrazu stojí slovo `yield`.

Vydávací generátorová funkce

Vydávací generátorová funkce obsahuje slovo **`yield`** na levé straně výrazu. Když takovou funkci vyvoláme, nevrací hodnotu jako v příkazu `return`, ale *vrátí objekt generátoru*. Dále již nevoláme funkci, ale voláme generátor příkazem (funkcí) `next()`. Mezi voláními si generátor pamatuje hodnoty lokálních proměnných.

Příklad – generování a získávání položek posloupnosti

```
def gener_funkce(seq):  
    for i in range(len(seq)):  
        yield seq[i] # yield na levé straně výrazu je vlastnost generátoru
```

Jednorázovým voláním funkce vytvoříme generátor a z něj příkazem `next()` postupně získáváme generované hodnoty jako z iterátoru.

```
generator = gener_funkce((1, "a", 3, "c", 5, "e")) # z položek n-tice  
next(generator) # 1  
next(generator) # a  
next(generator) # 3
```

Výraz se slovem `yield` vrátí na zavolání příkazu `next()` *další hodnotu* ze vstupní posloupnosti a zapamatuje si pozici v sekvenci pro příští volání příkazu `next()`.

Volání příkazu `next()` můžeme zabalit do cyklu `for-in` a vytiskneme tak všechny položky:

```
generator = gener_funkce((1, "a", 3, "c", 5, "e")) # z položek n-tice  
for x in generator:  
    print(x, end=" ")  
  
1 a 3 c 5 e
```

Příklad – geometrická řada

Generátorová funkce se nemusí omezovat jen na kopírování hodnot posloupnosti, ale může je upravovat nebo vynechávat, anebo generovat vlastní data. Následující funkce generuje *nekonečnou* geometrickou posloupnost $\frac{1}{2^n}$, pro $n = 1, 2, \dots$ podle rekurentního vzorce

$$n_0 = 1, n_1 = \frac{n_0}{2}, \dots, n_k = \frac{n_{k-1}}{2}, \dots$$

```
def gen_geom():
    n = 1
    while True: # neomezený cyklus
        yield n/2 # výpočet zde končí a vrací n/2
        n = n/2   # příští volání next() začne zde a opakuje cyklus
```

```
gg = gen_geom() # vytvoření generátoru geometrické posloupnosti
```

V následujícím *konečném* cyklu se vypočítá součet 20 prvních členů: $s = \sum_{n=1}^{20} \frac{1}{2^n}$.

```
s = 0
for n in range(1, 21): # od 1 do 20
    s += next(gg) # přičtu další člen geom. posloupnosti
```

```
print("suma =", s) # suma = 0.9999990463256836
```

Poznámka 1: Je známo, že součet nekonečné geometrické řady $s = \sum_{n=1}^{\infty} \frac{1}{2^n}$ je přesně 1.

Poznámka 2: Jako příklad můžeme tentýž součet vypočítat také pomocí *rekurze*.

```
def sum_geom(k, s): # od k sestupně do 1
    if k < 1: # na konci rekurze vrátím celkový součet
        return s
    s = (s + 1) / 2 # během rekurze se přičítají poloviny
    return sum_geom(k-1, s) # na konci kroku rekurze vrátím částečný součet
```

```
print("suma =", sum_geom(k=20, s=0)) # suma = 0.9999990463256836
```

Nebo také nejjednodušeji a nejrychleji *přímo cyklem* podle vzorce

$$s_0 = 1/2, s_1 = s_0 + 1/2, \dots, s_n = s_{n-1} + 1/2, \dots$$

```
def sum_geom(k, s):
    for n in range(k): # od 0 do k-1
        s = (s + 1) / 2
    return s
```

```
print("suma =", sum_geom(k=20, s=0)) # suma = 0.9999990463256836
```

Přijímací generátorová funkce

V generátorové vydávací funkci se slovo `yield` vyskytuje na levé straně výrazu. Slovo `yield` je ale možné použít i na *pravé straně* přiřazovacího příkazu, kde však neprodukuje výstup, ale naopak, *přijímá vstup* přicházející z vnějšího příkazu `send()` nebo `close()`.

Příklad – tisk přijatých hodnot

Následující přijímací generátorová funkce `echo()` po vyvolání vytvoří přijímač `e` – objekt třídy `Generator`. Přijímač `e` po inicializaci funkcí `next(e)` pak v neomezeném cyklu přijímá hodnotu z příkazu `send()` a tiskne ji. Tím přijatou hodnotu konzumuje a nepředává ji nikam dál. Po přijetí příkazu `close()` cyklus končí.

```
def echo():
    try:
        while True: # neomezený cyklus
            x = (yield) # yield na pravé straně přijme hodnotu a dosadí ji do x
            print("x =", x) # vytiskne přijatou hodnotu
    except GeneratorExit: # výjimka nastane po přijetí příkazu close()
        print("Konec příjmu")
```

```
e = echo() # vytvořím přijímač
next(e) # inicializace vrátí None, ale přijímač nic netiskne
```

Voláním funkce `echo()` vytvoříme *přijímač* – objekt `e` třídy `Generator`, který musíme nejprve *nastartovat* či *inicializovat* buď příkazem `next(e)`, nebo příkazem `send(None)`; oba vracejí hodnotu `None`. Příkaz `next(e)` už pak nevydáváme, protože by vracel stále stejnou hodnotu.

Při *prvním* přijetí hodnoty `None` se výpočet zastaví ve výrazu `yield` a čeká. Teprve po dalším přijetí hodnoty se provede přiřazovací příkaz a hodnota se dostane do proměnné `x` na levé straně.

Přijímači pak můžeme libovolně dlouho posílat v příkazech `send()` zvolené hodnoty, které se dostávají do proměnné `x`, odkud je tiskneme. Nakonec uzavřeme přijímač příkazem `close()`. Ten vyvolá výjimku `GeneratorExit`.

```
e.send(6) # přijímač tiskne x = 6
e.send("ahoj") # přijímač tiskne x = ahoj
...         příkazy send() můžeme posílat libovolně dlouho
e.close() # přijímač tiskne Konec příjmu
```

Volání metody `send()` můžeme umístit např. do cyklu `for-in`, který si sám skrytě hlídá výjimku `StopIteration` – konec iterace – a provede její příkaz.

```
for i in range(3):
    e.send(i) # tiskne postupně x = 0 až x = 2
e.close() # přijímač tiskne Konec příjmu
```

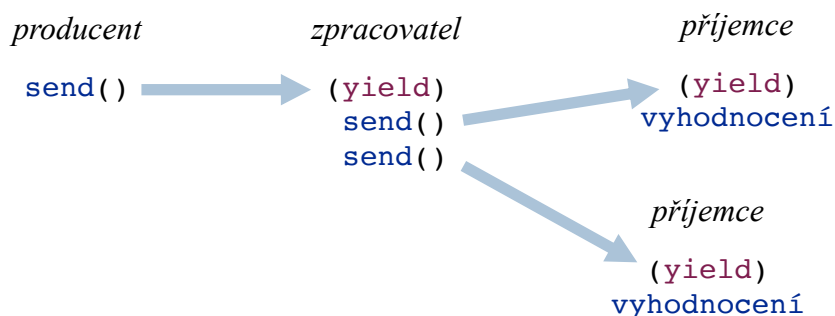
Poznámka: Příkazy `send()` a `close()` jsou metody třídy `Generator`.

Korutiny – pipeline (potrubí)

Generátorová funkce může být zároveň přijímací i vydávací. Pro příjem obsahuje přiřazovací příkaz `yield` a pro výdej příkaz `send()`. Taková funkce se nazývá *korutina* (*coroutine*). Nevyskytuje se osaměle, ale společně s jinými korutinami, které si vzájemně předávají data. Každá z nich má svůj specifický, co nejjednodušší úkol při úpravě nebo vyhodnocení dat, tak aby zpracování bylo co nejpřehlednější a co nejméně náchylné k chybám. Řazení korutin se také říká *pipeline* neboli *potrubí*.

Poznámka: Pod názvem Coroutines je v dokumentaci Pythonu uveden *asynchronní* způsob komunikace mezi funkcemi s použitím modulu `asyncio` a příkazu `await`. Zde však probíráme původní koncept korutin, který je *synchronní*.

Příklad – příjem výrazem `yield` a výdej příkazem `send()`



V příkladu jsou v potrubí tři funkce: producent, zpracovatel a příjemce.

Producent má dva parametry: vstupní *n*-tici a generátor cílové korutiny. Přijímá data ze vstupní *n*-tice a posílá jednotlivé položky postupně cílové korutině – zpracovateli příkazem `send()`. Producent není generátorová funkce, ale slouží jako první zdroj dat pro další korutiny.

```
def producent(n_tice, cil_kor):
    for i in (n_tice):
        cil_kor.send(i) # posílá položky cílové korutině – zpracovateli
```

Zpracovatel má dva parametry – generátory dvou cílových korutin. V nekonečném cyklu přijímá jednotlivé položky od producenta – `(yield)` a posílá příjemcům dvojice údajů (text a hodnotu); první cílové korutině posílá jako hodnoty všechny položky nezměněné a druhé cílové korutině posílá jen celá čísla (zdvojnásobená).

```
def zpracovatel(cil_kor1, cil_kor2): # cílí na dvě korutiny – příjemce
    while True:
        obj = (yield)
        # posílá dvojice dvěma příjemcům
        if type(obj) == int:
            cil_kor1.send(("První", obj)) # čísla prvnímu příjemci
        else:
            cil_kor2.send(("Druhý", obj)) # ostatní druhému příjemci
```

Příjemce má jeden parametr – výstupní soubor. Přijímá v nekonečném cyklu dvojice údajů od zpracovatele – (`yield`). Hodnoty ve znakovém tvaru oddělené čárkou запиše příjemce do souboru daného argumentem.

```
def příjemce(outfile): # zapisuje do souboru
    with outfile:
        ch = ""
        while True:
            x, y = (yield) # rozbalení přijaté dvojice
            print(f"{x} přijal hodnotu {y}.")
            outfile.write(ch + str(y) # zápis čísla do souboru
            ch = "," # oddělovací čárka
```

Spuštění příkladu

- Začínáme vytvořením dvou různých generátorů příjemce, zpracovatel je použije ve svých argumentech. Oba tyto generátory inicializujeme příkazem `next()`.

```
file1 = open("fcisla.t", "w")
pri1 = příjemce(file1) # vytvořím generátor 1. příjemce
next(pri1) # inicializuji generátor 1. příjemce
```

```
file2 = open("fznaky.t", "w")
pri2 = příjemce(file2) # vytvořím generátor 2. příjemce
next(pri2) # inicializuji generátor 2. příjemce
```

- Dále vytvoříme generátor zpracovatele s dvěma argumenty – generátory příjemců – a inicializujeme jej příkazem `next()`.

```
zpr = zpracovatel(pri1, pri2) # vytvořím generátor zpracovatele
next(zpr) # inicializuji generátor zpracovatele
```

- Producent není generátorová funkce (neobsahuje `yield`), proto jej rovnou spustíme s dvěma argumenty – n-tice a generátor zpracovatele.

```
producent((1, "a", 3, "c", 5, "e"), zpr) # spustím funkci producenta
```

Příjemce tiskne do standardního výstupu:

```
První přijal hodnotu 1.
Druhý přijal hodnotu a.
První přijal hodnotu 3.
Druhý přijal hodnotu c.
První přijal hodnotu 5.
Druhý přijal hodnotu e.
```

a zapisuje do výstupních souborů:

fcisla.t	fznaky.t
1,3,5	a,c,e

Příklad – postupné zpracování čísel z textového souboru

Budeme mít tři korutiny, které si vzájemně předávají čísla vybraná z textového souboru:

- výběr čísel ze souboru čtením *znak po znaku* – producent,
- převod čísel ze znakového tvaru na typ `float` s kontrolou – zpracovatel,
- vyhodnocení `float` čísel – příjemce

```
def vyber_cisel(f, dalsi_korutina): # obyčejná funkce – využívá příkaz send()
    cislo = ""
    ch = f.read(1) # čtu první znak souboru f
    next(dalsi_korutina) # nejdřív musím vydat next() – jen jednou
    while ch != "": # dokud není konec souboru, zpracuji přečtený znak
        if not (ch.isspace() or ch == ','): # vynechám whitespace a čárku
            cislo += ch # přidám znak k číslu
        else: # whitespace nebo čárka
            dalsi_korutina.send(cislo) # pošlu hodnotu příjemci
            cislo = "" # vyprázdním staré číslo pro střádání nového
        ch = f.read(1) # čtu další znak souboru
    # na konci cyklu pošlu poslední hodnotu
    dalsi_korutina.send(cislo) # pošlu poslední řetězec "číslo" příjemci
    dalsi_korutina.close() # a uzavřu příjemce
```

```
def prevod_na_float(dalsi_korutina): # příjemce dat
    next(dalsi_korutina) # musím vydat next() – jen jednou
    while True:
        try:
            zn_cis = (yield) # čtu další řetězec – znakové číslo
            if zn_cis is None or zn_cis == "":
                continue # vynechám objekt None a prázdné znakové číslo
            fl = float(zn_cis) # pokus o přeměnu řetězce na číslo
            dalsi_korutina.send(fl) # pošlu hodnotu příjemci
        except ValueError:
            print(f" Řetězec {repr(zn_cis)} nelze převést na číslo.")
        except GeneratorExit: # po příkazu close()
            dalsi_korutina.send(fl) # pošlu poslední hodnotu příjemci
            dalsi_korutina.close() # a uzavřu příjemce
            break # končím nekonečný cyklus
```

```
def vyhodnoceni_cisel(maximum):
    while True:
        try:
            fl = (yield) # čtu další číslo
            if fl > maximum:
                maximum = fl # počítám průběžné maximum
        except GeneratorExit: # po příkazu close()
            print("maximum =", maximum)
            break # končím nekonečný cyklus
```

Obsah souboru `cisla.txt`:

```
23.5, +32,  
4_3.5_55,20, *19,  
6,4_2.
```

Hlavní program začíná vytvořením korutin z generátorových funkcí, nejprve z poslední, přijímací, protože další ji používá jako argument.

```
if __name__ == "__main__":  
    # vytvoření generátorových funkcí:  
  
    # vyhodnocovatel maxima 'float' čísel  
    gen_vyhod = vyhodnoceni_cisel(float("-inf")) # argument: nejmenší číslo  
    # přijímač vybraných čísel a vysílač 'float' čísel  
    gen_prevod = prevod_na_float(gen_vyhod) # argument: následující korutina  
  
    # spuštění producenta s následující korutinou jako argumentem  
  
    f = open("cisla.txt")  
    vyber_cisel(f, gen_prevod) # argumenty: soubor, následující korutina
```

Vytiskne se

```
23.5 +32 4_3.5_55 20 *19  
Chyba v 2. řádku, 3. prvku ' *19': could not convert string to float: ' *19'  
6 4_2 maximum = 43.555
```

Poznámka: Inicializace korutin probíhá uvnitř předchozích korutin, není tedy v hlavním programu.

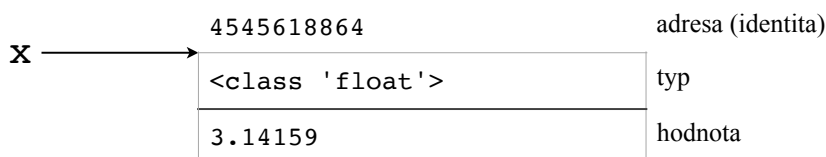
Objekty a jména

Python ukládá data do paměti jako objekty. Každý objekt má *adresu (identitu)*, *typ* a *hodnotu*. Objekt může měnit hodnotu, ale nemůže měnit adresu ani typ. Na objekty odkazují *jména*. Jméno neobsahuje hodnotu objektu, ale *ukazatel* na objekt (*adresu* objektu). Jméno je tak vázané (bound) na objekt. Jménům se často říká proměnné, zvláště když se vyskytnou na levé straně přiřazovacího příkazu.

Adresa (identita) objektu se zjistí pomocí funkce `id( jméno )`, typ objektu se zjistí pomocí funkce `type( jméno )`.

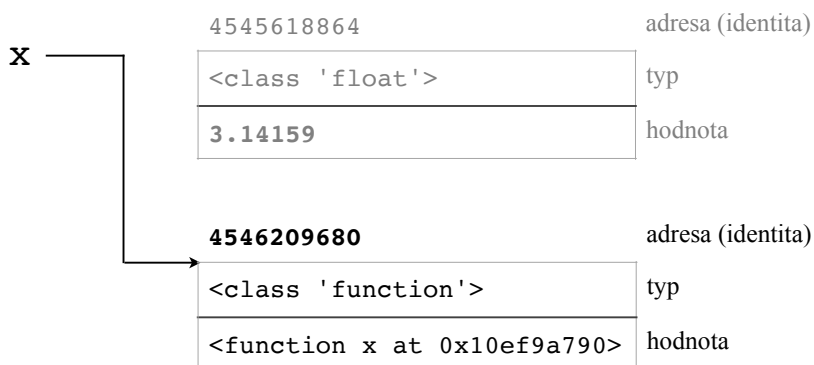
```
x = 3.14159
print("id(x):", id(x))      # id(x): 4545618864
print("type(x):", type(x)) # type(x): <class 'float'>
print(x)                   # 3.14159
```

V paměti se z literálu `3.14159` vytvoří nový objekt. Je mu přidělena adresa, typ a hodnota. Pak se vytvoří jméno `x`, které obsahuje ukazatel na tento objekt (adresu objektu).



Když jménem `x` pojmenujeme jiný (nový) objekt, ztratí se jeho dosavadní ukazatel a jméno získá jiný ukazatel.

```
def x():
    pass
print("id(x):", id(x))      # id(x): 4546209680
print("type(x):", type(x)) # type(x): <class 'function'>
print(x)                   # <function x at 0x10ef9a790>
```

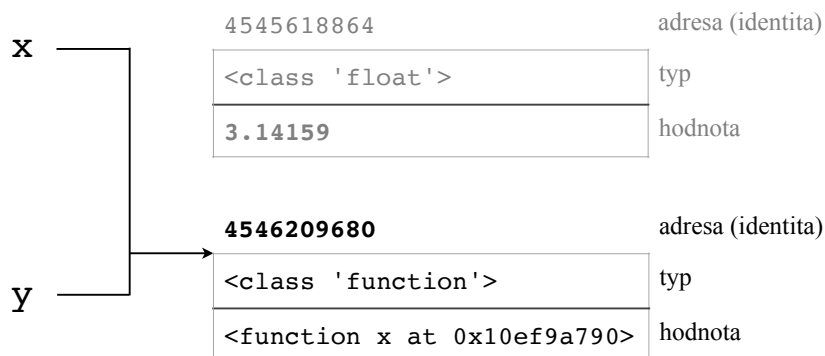


Ted' jméno `x` ukazuje na jiný objekt – s jinou adresou, jiným typem a jinou hodnotou. Na původní objekt neukazuje žádné jméno. Ten je interpretem Pythonu v určitých časových intervalech vymazán z paměti pomocí procedur *čítání odkazů* (reference counting) a *sběru odpadu* (garbage collection), viz <https://devguide.python.org/internals/garbage-collector/>.

Vytvoříme nové jméno `y` tak, že mu přiřadíme jméno `x`.

```
y = x
print("id(y):", id(y))      # id(y): 4546209680
print("type(y):", type(y)) # type(y): <class 'function'>
print(y)                   # <function x at 0x10ef9a790>
```

Jméno `y` bude ukazovat na stejný objekt jako jméno `x`. Je to vidět podle stejného čísla identity.



Můžeme to ověřit porovnáním na rovnost.

```
print(y == x) # True
print(y is x) # True
```

Obecné vlastnosti objektů

Hodnota složitějších objektů v našich ukázkách je jen označení. Skutečnou hodnotou objektu je jeho obsah, kterým může být velmi složitá kombinace dalších objektů, zpravidla dat a příkazů (například v objektu funkce).

Některé objekty mají *neměnný* (immutable) vnitřek, např. *čísla*, *řetězce*, *n-tice*, *množiny* (sets).

Jiné objekty mají vnitřky *měnitelné* (mutable), např. *seznamy*, *slovníky*. Samotný objekt jako celek je neměnný.

Některé objekty obsahují *odkazy* na jiné objekty, např. *n-tice*, *seznamy*, *slovníky*. Nazývají se *kontejnery* (containers).

Podrobnosti jsou uvedeny v dokumentaci:

<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>.

Jména – identifikátory

Python v oficiální dokumentaci nezná pojem proměnná (variable), ale mluví o identifikátorech (identifiers) a o *jménech* (names). Jméno může být libovolně dlouhé a může se skládat z písmen A až Z, a až z, podtržítka `_` a číslic 0 až 9. Nesmí však začínat číslicí. Od verze Python 3.0 jsou povoleny některé znaky Unicode, mezi nimi i česká diakritika. Přesně je to popsáno v [dokumentaci](#).

Pro názvy proměnných nelze používat *klíčová slova* (keywords), ani jména z *rezervovaných tříd*.

Klíčová slova (keywords)

False	await	else	import	pass	None	break
except	in	raise	True	class	finally	is
return	and	continue	for	lambda	try	as
def	from	nonlocal	while	assert	del	global
not	with	async	elif	if	or	yield

Rezervované třídy jmen

`_` V neinterakčním režimu je to obyčejné jméno.
V interakčním režimu obnovuje výsledek operace:

```
>>> 5+5
10
>>> _ +5
15
>>>
```

`_jméno` Chráněné (protected) jméno v definici třídy (class).
V příkazu `from modul import *` se takové jméno neimportuje.

`__jméno` Privátní (private) jméno v definici třídy (class).

`__jméno__` Dvěma podtržítky před a za jménem se označují některá jména definovaná v systému a v definici třídy (class).

Vytváření (definice) jmen

Jména se v programu vytvářejí (vážou se k objektům) následujícími způsoby.

- přiřazovacím příkazem `x = ...`
- definicí funkce `def x ...`
- parametrem funkce `( x, y, ... )`
- definicí třídy `class x ...`
- příkazem `for x in ...`
- příkazem `with ... as x`
- příkazem `except ... as x`
- příkazem `import`

Jméno by mělo být vytvořeno (definováno) před jeho použitím.

Prostory jmen (namespaces)

Každé jméno v programu patří do nějakého *prostoru jmen*. Tento prostor je dán kontextem, v němž jméno vytvoříme (definujeme).

Lokální prostor jmen

Jméno definované *ve funkci* patří do lokálního prostoru funkce a nazývá se lokální jméno. Patří tam i jména argumentů ve volání funkce.

Nelokální (obalový) prostor jmen

Jméno definované *v obalové funkci*, tj. ve funkci, která obsahuje nejbližší vnořenou funkci, patří do *nelokálního* prostoru jmen a nazývá se nelokální jméno vzhledem ke vnořené funkci.

Globální prostor jmen – modul

Jméno definované v modulu mimo všechny funkce patří do *globálního* prostoru jmen a nazývá se globální proměnná v rámci modulu.

Vestavěný prostor jmen (built-in namespace)

Vestavěný prostor obsahuje *jména všech standardních objektů*. Jména z tohoto prostoru jsou přímo přístupná z jakéhokoliv místa programu, nejsou-li ovšem překryta přímou definicí.

Jsou to jména

- standardních typů, např. `int`, `float`, `str`, `list`, `dict` aj.
- výjimek, např. `Exception`, `ValueError`, `StopIteration` aj.
- standardních funkcí, např. `max()`, `repr()`, `range()` aj.
- standardních konstant, např. `False`, `True`, `None` aj.

Jména standardních objektů se získají voláním standardní funkce `dir()`

```
dir(__builtins__)
```

Popisy standardních objektů se získají voláním funkce `help()`

```
help("builtins")
```

Příklad

```
x = 300 # globální proměnná
def f():
    print("globální:", x) # globální proměnná x
    y = 200 # nelokální proměnná y vzhledem k funkci g
    def g():
        print("nelokální z f:", y)
        z = 100 # lokální proměnná z
        print("lokální v g:", z)

    g() # volám vnořenou funkci
```

```
f() # volám obalovou funkci, aby se mohla volat vnořená funkce g
```

```
globální: 300
nelokální z f: 200
lokální v g: 100
```

Hodnotu globální proměnné můžeme zjišťovat (například tisknout) v každém místě programu. Podobně můžeme zjišťovat hodnotu nelokální proměnné i ve funkcích vnořených do obalové funkce.

Poznámka 1: Globální a nelokální proměnné je vždy možné zjišťovat (číst), ale není možné je měnit (přepisovat) jinak než přiřazením jiného objektu. Přiřazením jiného objektu se proměnná jen definuje jako lokální.

Poznámka 2: Funkce `print()` je vestavěná, proto je dosažitelná z každého místa programu.

Klíčová slova **global** a **nonlocal**

Když globální proměnnou označíme klíčovým slovem **global**, můžeme její hodnotu nejen zjišťovat, ale i *měnit*. Podobně se chová nelokální proměnná, kterou deklarujeme s klíčovým slovem **nonlocal**. Příslušná deklarace musí být umístěna na začátku těla funkce a před definicí proměnné.

```
x = 300 # globální proměnná
def f():
    global x
    x = x + 1 # změna globální proměnné x ve funkci f
    print("globální:", x) # čtení globální proměnné x
    y = 200 # nelokální proměnná y vzhledem k funkci g
    def g():
        global x
        x = x + 1 # změna globální proměnné x ve funkci g
        nonlocal y
        y += 1 # změna nelokální proměnné y ve funkci g
        print("nelokální v f:", y) # čtení nelokální proměnné y
        z = 100 # lokální proměnná z ve funkci g
        print("lokální v g:", z)

    g() # volám vnořenou funkci g
```

`f()` # volám obalovou funkci f, aby se mohla volat vnořená funkce g

```
globální v f: 301
globální v g: 302
nelokální z f: 201
lokální v g: 100
```

Jak Python hledá jména

Odkazuje-li program (příkaz) v určitém místě na nějaké jméno, třeba `x`, hledá je Python nejprve ve vlastním jmenném prostoru. Když je najde, použije je. Když je nenajde, hledá v dalším jmenném prostoru podle následujícího pořadí.

- lokální
- nelokální
- globální – modul
- vestavěný (built-in)

Když najde jméno v některém z těchto prostorů, použije je. Když jméno nenajde nikde, hlásí chybu `"NameError: name 'x' is not defined"`.

Jméno musí být vytvořeno (definováno) před jeho použitím.

Volba jmen

V programu by se neměla používat jména *standardních objektů* pro jiné účely. Třeba jména `int`, `float` a `str` kromě typů jsou také jmény standardních funkcí `int()`, `float()` a `str()`. Volba takových jmen je sice možná, ale může působit těžkosti při výpočtu.

Například následující úryvek programu převede číslo na řetězec bez problému.

```
x = 3.14159
x = str(x)    # převod na řetězec
print(x)      # 3.14159
```

Změníme-li však jméno `x` na `str`, dojde k chybě:

```
str = 3.14159
str = str(str) # převod na řetězec
print(str)     # chyba: TypeError: 'float' object is not callable
```

Moduly

Každý soubor s příponou `.py` obsahující příkazy a definice jazyka Python je *modul*. Modul má své jméno, totožné se jménem souboru (bez přípony `.py`), které lze při výpočtu zjistit z globální proměnné `__name__`. Výjimkou je *hlavní modul*, kterým začíná program. V proměnné `__name__` nemá jméno svého souboru, ale jméno `__main__`.

Jména z modulu mohou být *importována* do hlavního modulu nebo do jiných modulů.

Příklad importu

Máme dva soubory – moduly ve stejném adresáři: `vypocty.py` jako hlavní modul a `vypocetni_operace.py` jako *importovaný* modul se dvěma definicemi funkcí `plus()` a `krat()`, a s příkazem `print()` k vytištění jména modulu.

Importovaný modul `vypocetni_operace.py`

```
def plus(a, b):
    c = a + b
    print(c)

def krat(a, b):
    print(a * b)

plus(1, 2) # 3
print("__name__:", __name__) # __name__: vypocetni_operace
```

Příkazy `plus(1,2)` a `print()` jsou zapsané mimo definice funkcí a provedou se vždy při spuštění modulu. První příkaz vytiskne číslo 3, druhý příkaz vytiskne jméno svého modulu. Definice funkcí `plus()` a `krat()` se přitom neprovádějí.

Hlavní modul **vypocty.py**

Před jmény volaných funkcí musí být zapsáno *jméno importovaného modulu s tečkou*.

```
import vypocetni_operace

vypocetni_operace.plus(2, 3) # jméno importu s tečkou před jménem funkce
vypocetni_operace.krat(2, 3)
print("__name__:", __name__)
```

Při spuštění hlavního modulu se nejprve provede *příkaz import* a v něm dva příkazy: `plus(1,2)` a `print("__name__:", __name__)`. Teprve potom se volají jeho funkce. Výsledek je následující.

```
3
__name__: vypocetni_operace
5
6
__name__: __main__
```

Máme-li nějaké vícenásobně používané funkce, umístíme jejich definice do modulu, v němž nebudou žádné volné příkazy (tj. příkazy v rámci hlavního modulu `__main__`). V našem příkladu bychom zmíněné dva příkazy z modulu `vypocetni_operace` vyřadili.

Formy importu

Přejmenování importovaného modulu

Používá se pro zkrácení dlouhého jména modulu.

```
import vypocetni_operace as op
```

```
op.plus(2, 3)
```

```
op.krat(2, 3)
```

Vyjmenování použitých funkcí

```
from vypocetni_operace import plus, krat
```

```
plus(2, 3)
```

```
krat(2, 3)
```

Použití hvězdičky pro použití všech funkcí z importu

```
from vypocetni_operace import *
```

```
plus(2, 3)
```

```
krat(2, 3)
```

Několik modulů v řádku

```
import os, math
```

Získání informací o modulech

Existuje mnoho modulů obsahujících definice obecně užitečných funkcí. Jejich seznamy i s popisem lze vypisovat pomocí funkce `help()`.

```
print(help('builtins')) # vestavěhé funkce, výjimky a další objekty
print(help('open'))    # popis funkce open z modulu builtins
print(help('os'))      # práce s adresáři a soubory jako celky
print(help('sys'))     # přístup k objektům používaným v interpretu Pythonu
print(help('math'))    # matematické funkce
atd.
```

Jak Python hledá moduly

Když je spuštěn program, hledá Python moduly postupně v seznamu adresářů obsaženém v proměnné `sys.path`. Ten je sestaven z následujících zdrojů:

- adresář, kde je program spuštěn (v případě *interakčního režimu* je to běžný adresář uživatele operačního systému),
- seznam adresářů obsažených v systémové proměnné `PYTHONPATH`,
- seznam adresářů v instalačním adresáři Pythonu.

Následující výpisy jsou provedeny z prostředí PyCharm.

```
import sys
for p in sys.path:
    print(p)
```

macOS

```
/Users/vzupka/PycharmProjects/vyber_temat
/Users/vzupka/PycharmProjects/vyber_temat
/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9.zip
/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9
/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/lib-dynload
/Users/vzupka/PycharmProjects/vyber_temat/venv/lib/python3.9/site-packages
```

Linux

```
/home/vzupka
/home/vzupka
/Users/vzupka/PycharmProjects/vyber_temat
/home/vzupka/PycharmProjects/vyber_temat
/usr/lib/python3.8.zip
/usr/lib/python3.8
/usr/lib/python3.8/lib-dynload
/usr/local/lib/python3.8/dist-packages
/usr/lib/python3/dist-packages
```

Windows

```
C:\Users\vzupka\PycharmProjects\vyber_temat
C:\Users\vzupka\PycharmProjects\vyber_temat
C:\Program
Files\WindowsApps\PythonSoftwareFoundation.Python.3.9_3.9.496.0_x64__qbz5n2kfra8p0\python39.zip
C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.9_3.9.496.0_x64__qbz5n2kfra8p0\DLLs
C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.9_3.9.496.0_x64__qbz5n2kfra8p0\lib
C:
\Users\vzupka\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9_qbz5n2kfra8p0
C:\Users\vzupka\PycharmProjects\vyber_temat\venv
C:\Users\vzupka\PycharmProjects\vyber_temat\venv\lib\site-packages
```

Spuštění modulu z příkazového řádku

macOS

```
export PYTHONPATH=$PYTHONPATH:/Users/vzupka/PycharmProjects/vyber_temat  
python3 -m vypocty
```

Linux

```
export PYTHONPATH=$PYTHONPATH:/home/vzupka/PycharmProjects/vyber_temat  
python3 -m vypocty
```

Windows

```
set PYTHONPATH=C:\Users\vzupka\PycharmProjects\vyber_temat  
python3 -m vypocty
```

```
3  
__name__: vypocetni_operace  
5  
6  
__name__: __main__
```

Třídění (sorting)

Tříděním se zde rozumí uvedení nějaké posloupnosti objektů do uspořádaného stavu pomocí porovnání jejich hodnot.

Python má dva standardní způsoby pro třídění. Standardní funkci `sorted()` použitelnou na *posloupnosti* a na *slovníky* a metodu `sort()` použitelnou *jen na seznamy*.

Třídění sekvencí

Standardní funkce `sorted()` třídí sekvenci *do nového seznamu*, který vrací. Volání funkce je definováno takto (s nejvýše dvěma klíčovými argumenty):

```
sorted(iterable, /, *, key=None, reverse=False)
```

iterable tříděný objekt schopný iterace (např. posloupnost, textový soubor s řádky)
key funkce s jedním argumentem, která získává porovnávací klíč z každého prvku
 tříděného objektu; `None` znamená přímé porovnání hodnot
reverse obrátit pořadí

```
s = sorted((7, 2, 3, 1, 4)) # třídí n-tici do seznamu
print(s) # [1, 2, 3, 4, 7]
s = sorted(s, reverse=True) # obrácené pořadí
print(s) # [7, 4, 3, 2, 1]
```

Setřídění klíčů slovníku do seznamu – podle pořadového čísla Unicode.

```
s = sorted({'z': 122, 'x': 120, 'č': 269, 'a': 97, 'ř': 345})
print(s) # ['a', 'x', 'z', 'č', 'ř']
```

Třídění seznamu řetězců podle Unicode.

```
s = sorted("My tři králové jdeme k vám".split())
print(s) # ['My', 'jdeme', 'k', 'králové', 'tři', 'vám']
```

Třídění s metodou `lower()` určující převod do malých písmen v argumentu `key`.

```
s = sorted("My tři králové jdeme k vám".split(), key=str.lower)
print(s) # ['jdeme', 'k', 'králové', 'My', 'tři', 'vám']
```

Třídění seznamu řetězců podle jejich délky s funkcí `len()`.

```
s = sorted(["ccc", "a", "bb"], key=len)
print(s) # ['a', 'bb', 'ccc']
s = sorted(["ccc", "a", "bb"], key=len, reverse=True)
print(s) # ['ccc', 'bb', 'a']
```


Třídění slovníku

```
cenik = {"jablka": 50.5, "hrušky": 67.00, "švestky": 30.99,  
        "broskve": 85.00,}
```

Nový slovník bude mít klíče seřazené vzestupně podle velikosti *hodnot*.

```
keys_by_values = sorted(cenik, key=cenik.get)  
print(keys_by_values) # ['švestky', 'jablka', 'hrušky', 'broskve']  
dict_by_values = dict() # prázdný nový slovník  
for k in keys_by_values:  
    dict_by_values[k] = cenik[k] # hodnoty ke klíčům v nových položkách  
print(dict_by_values)  
  
{'švestky': 30.99, 'jablka': 50.5, 'hrušky': 67.0, 'broskve': 85.0}
```

Nový slovník bude seřazen podle velikosti *klíčů*.

```
keys_by_keys = sorted(cenik)  
print(keys_by_keys) # ['broskve', 'hrušky', 'jablka', 'švestky']  
dict_by_keys = dict() # prázdný nový slovník  
for k in keys_by_keys:  
    dict_by_keys[k] = cenik[k] # hodnoty ke klíčům v nových položkách  
print(dict_by_keys)  
  
{'broskve': 85.0, 'hrušky': 67.0, 'jablka': 50.5, 'švestky': 30.99}
```

Třídění seznamu na místě

Metoda **sort()** setřídí položky seznamu *na místě*.

```
s = [7, 2, 3, 1, 4]  
s.sort()  
print(s) # [1, 2, 3, 4, 7]
```

České třídění

Dosavadní příklady, v nichž se třídí řetězce, uspořádávají výsledky porovnáním číselných hodnot znaků Unicode. Toto porovnání však zvyklostem v národních jazycích nevyhovuje:

```
"zč" < "čz"  
"ch" < "h"
```

Proto se zavádí prostředí zvané *locale* (místní). Ani to však v Pythonu nevyhovuje všem jazykům, například češtině. Existují knihovní moduly, v nichž se tento problém řeší, například zde:

<https://pypi.org/project/PyICU/>
<https://pypi.org/project/czech-sort/>

Modul `czech_sort` pro české třídění se instaluje příkazem

```
pip3.9 install czech-sort
```

Třídění seznamu podle české abecedy (včetně ch)

Oba následující způsoby jsou rovnocenné.

```
import czech_sort  
  
print(sorted(['ě', 'c', 'e', 'č', 'é', 'ch', 'h'], key=czech_sort.key))  
  
# ['c', 'č', 'e', 'é', 'ě', 'h', 'ch']  
  
print(czech_sort.sorted({'z': 122, 'x': 120, 'č': 269, 'a': 97, 'ř': 345}))  
  
# ['a', 'č', 'ř', 'x', 'z']
```

Třídění slovníku podle české abecedy

Nový slovník bude seřazen podle hodnot klíčů v české abecedě.

```
chars = {'z': 122, 'x': 120, 'č': 269, 'a': 97, 'ř': 345}  
  
keys_by_keys = czech_sort.sorted(chars)  
print(keys_by_keys)  
dict_by_keys = dict() # prázdný nový slovník  
for k in keys_by_keys:  
    dict_by_keys[k] = chars[k] # hodnoty ke klíčům v nových položkách  
print(dict_by_keys)  
  
# {'a': 97, 'č': 269, 'ř': 345, 'x': 120, 'z': 122}
```

Národní prostředí (locale)

Národní prostředí znamená úpravu některých údajů proti těm, které jsou zavedeny přímo v jazyku. K tomu je zapotřebí importovat modul `locale`. Dokumentace k němu je na stránce <https://docs.python.org/3/library/locale.html>.

```
import locale
```

Nastavení českého prostředí

K dalšímu použití je třeba nastavit konkrétní národní prostředí. České prostředí je dáno kódem `'cs_CZ'` (standard Language Code Identifier (LCID) Reference).

```
locale.setlocale(locale.LC_ALL, 'cs_CZ')
```

Následujícím programem lze vypsat jednotlivé zvláštnosti českého prostředí.

```
for key, value in locale.localeconv().items():
    print(f"{key:18}= '{value}'")

int_curr_symbol    = 'CZK '
currency_symbol    = 'Kč'
mon_decimal_point  = ','    mon = monetary
mon_thousands_sep = ' '
mon_grouping       = '[3, 3, 0]'
positive_sign      = ''
negative_sign      = '-'
int_frac_digits    = '2'
frac_digits        = '2'
p_cs_precedes      = '0'    currency symbol precedes positive value
p_sep_by_space     = '1'    currency symbol separated from positive value by a space
n_cs_precedes      = '0'    currency symbol precedes negative value
n_sep_by_space     = '1'    currency symbol separated from negative value by a space
p_sign_posn        = '1'    position of sign of positive value
n_sign_posn        = '1'    position of sign of negative value
                        (1=sign precedes value and currency symbol)
decimal_point      = ','
thousands_sep      = ' '
grouping           = '[3, 3, 0]'
```

Ukázky výstupu čísel v českém prostředí

```
print(locale.atoi('3,5'))          # 3.5
print(locale.atoi('3_555_555'))     # 3555555
print(locale.str(-13.44444))        # -13,44444
print(locale.format_string('%10.2f', -321543.65)) # -321543,65
print(locale.format_string('%10.2f', 321543.65, grouping=True)) # 321 543,65
print(locale.currency(-987654.32))  # -987654,32 Kč
print(locale.currency(987654.32, grouping=True)) # 987 654,32 Kč
print(locale.currency(987654.32, grouping=True, international=True))
# 987 654,32 CZK
```

Uvnitř formátovaného řetězce u typů `g`, `G`, `n` čísla 8 a 3 za tečkou neznamenaají počet desetinných míst, ale maximální počet nenulových číslic zleva. Číslo 10 před tečkou znamená minimální počet vyhrazených pozic pro výstup. Kód `n` používá *locale* interně, je-li importováno. Kódy `g`, `G` je nepoužívají.

```
print(f'{987654.32:10.8n}') # 987 654,32
print(f'{54.32:10.3n}')    #          54,3

print(f'{987654.32:10.8G}') # 987654.32
print(f'{54.32:10.3g}')    #          54.3
```

Časové údaje

Práce s časovými údaji je soustředěna v několika modulech: *datetime*, *zoneinfo*, *calendar*, *time*. Probereme jen modul *datetime*, který je popsán na stránce <https://docs.python.org/3/library/datetime.html>. Obsahuje funkce potřebné k práci s *datem*, *denním časem* a *časovým razítkem* (až po mikrosekundy). Umožňuje k času přičítat nebo od něj odečítat *časový interval*. Také umožňuje zadat světovou *časovou zónu*.

```
from datetime import date, time, datetime, timedelta, timezone
```

Názvy v importu jsou názvy tříd (classes) s odpovídajícími metodami.

Zjištění dnešního data

```
dnes = date.today()
print("dnes =", dnes) # 2021-05-06
```

Zjištění současného časového razítka

```
nyni = datetime.now()
print(repr(nyni)) # datetime.datetime(2021, 5, 6, 16, 59, 14, 638315)
print(nyni) # 2021-05-06 16:59:14.638315
print(nyni.isoformat()) # 2021-05-06T17:25:24.126412
```

Zjištění denního času

```
print(time(nyni.hour, nyni.minute, nyni.second, nyni.microsecond))  
# 16:59:14.638315
```

Přičtení časového intervalu

```
timedelta(days=0, seconds=0, microseconds=0, milliseconds=0,  
           minutes=0, hours=0, weeks=0)
```

```
pozdeji = dnes + timedelta(days=+31) # plus 31 dnů  
print(pozdeji) # 2021-06-06
```

Stanovení světové časové zóny vzhledem k UTC (Universal Time Coordinated)

```
t = timezone(timedelta(hours=+1)) # zimní čas v ČR  
print(t.tzname(nyni)) # UTC+01:00
```

Formátovaný čas pro výstup

```
strftime(format)
```

```
print(nyni.strftime('%H.%M - %A %d %B %Y')) # 17.25 - čtvrtek 06 května 2021
```

Parametr *format* je formátovací řetězec se speciálními symboly začínajícími znakem %. Názvy odpovídají národnímu prostředí (locale). Neúplná tabulka slouží k první orientaci.

Symbol	Význam	Příklad
%a	Den v týdnu (zkratka)	po, út, ..., so, ne
%A	Den v týdnu (plný název)	pondělí, úterý, ..., sobota, neděle
%d	Den v měsíci (plné dvojčíslí)	01, 02, ..., 31
%e	Den v měsíci (bez vedoucí nuly)	1, 2, ..., 9, 10, ..., 31
%b	Měsíc (zkratka)	led, úno, ..., pro
%B	Měsíc (plný název)	leden, únor, ..., prosinec
%y	Rok (plné dvojčíslí)	01, 02, ..., 99
%Y	Rok (čtyřčíslí)	0001, 0002, ..., 2013, 2014, ..., 9998, 9999
%H	Hodina	01, 02, ..., 22, 23
%M	Minuta	00, 01, 02, ..., 59
%S	Sekunda	00, 01, 02, ..., 59
%f	Mikrosekunda	000000, 000001, ..., 999999